
Normalizador Estadístico Vectorizado

Comparación de rendimiento entre una implementación escalar y una implementación vectorizada (AVX2/SSE) en NASM x86-64

Steven Loaiza Valverde
Carné: 2017200859
stevenloaiza1@estudiantec.cr

Felipe Sánchez Segura
Carné: 2023083272
fel.sanchez@estudiantec.cr

Profesor: Dr. Ing. Ronny García Ramírez

Fecha de entrega: 1 de octubre de 2026

Resumen — El presente informe compara el rendimiento de un algoritmo de normalizador estadístico (media, varianza, mínimo, máximo y estandarización) implementado en la plataforma de NASM x86-64, teniendo en cuenta 2 versiones: escalar (SSE) y vectorizada (AVX2). Asimismo, estas versiones se verificaron numéricamente contra NumPy con tolerancia relativa 10^{-4} . Los posteriores análisis se efectuaron sobre un AMD Ryzen 5 7520U, el speedup medido fue no monótono el cual fue de $6,7\times$ a un máximo de $8,3\times$ y el mismo cayó a $3,7\times$ para un $N = 5 \times 10^7$ debido a un desplazamiento del cuello de botella de cómputo a ancho de banda de memoria (confirmado con el análisis en `perf stat`). Finalmente, la verificación se efectuó con GDB la cual verifiqué que la ejecución vectorizada coincide con el comportamiento diseñado.

Palabras clave — SIMD, AVX2, NASM, x86-64, vectorización, normalización estadística, speedup.

Índice

1. Introducción y objetivos del trabajo	2
1.1. Objetivos	3
2. Descripción del entorno de pruebas	4
2.1. Hardware	4
2.2. Comparación de extensiones vectoriales: SSE vs. AVX2	4
2.3. Software	5
3. Implementación escalar	6
3.1. Convención de llamada y prólogo/epílogo escalar	6
3.2. <code>sum_array</code>	8
3.3. <code>compute_stats</code>	9
3.4. <code>normalize_array</code>	11
4. Implementación vectorial	12
4.1. Justificación de la extensión vectorial elegida	13
4.2. Convención de llamada y prólogo/epílogo vectorial	14
4.3. <code>sum_array</code> vectorizado	15
4.4. <code>compute_stats</code> vectorizado	16
4.5. <code>normalize_array</code> vectorizado	20
4.6. Estrategia de reducción horizontal	22
4.7. Manejo del remanente (tail loop)	22
4.8. Alineación de memoria	22
5. Casos de prueba y verificación de correctud	23
6. Resultados de rendimiento	24
6.1. Tiempos de ejecución	24
6.2. Speedup	24
6.3. Análisis con <code>perf stat</code>	25
7. Evidencia de la sesión de depuración con GDB	27
7.1. Inspección de registro YMM tras <code>vaddps</code>	27
7.2. Inspección de memoria del arreglo de salida	30
8. Conclusiones, limitaciones y trabajo futuro	31
8.1. Conclusiones	31
8.2. Limitaciones observadas	32
8.3. Trabajo futuro	33
A. Referencia al diagrama de arquitectura (entregable 3.a)	34
B. Pasos para correr el proyecto	36
C. Link del GitHub	36
Referencias	37

1. Introducción y objetivos del trabajo

A lo largo de la historia los diversos procesadores de propósito general han mejorado su desempeño de forma casi automática, esto se debe al escalado en la tecnología de los semiconductores, puesto que cada una de las nuevas generaciones de transistores son mucho más pequeñas y al mismo tiempo estas generan una mayor velocidad y una mejora significativa en la eficiencia energética. Sin embargo, se debe resaltar que alrededor de la mitad de la década de los 2000 este crecimiento en la tecnología de semiconductores se estanco debido al fin del *escalado de Dennard* y a los distintos límites físicos y térmicos que tiene el silicio como elemento químico. Asimismo, esto obliga a las diversas industrias tecnológicas y de semiconductores a buscar un nuevo mecanismo, método o vía para poder obtener un rendimiento adicional, entre los cuales se empezó a utilizar las siguientes estrategias: en primer lugar en vez de un solo un núcleo cada vez más rápido, los diversos fabricantes optaron por integrar múltiples núcleos y en segundo lugar se empezaron a emplear unidades capaces de operar sobre un conjunto de datos con solo el uso de una sola instrucción. Esta segunda estrategia dio origen al paradigma **SIMD** (*Single Instruction, Multiple Data*), puesto que como su nombre lo refleja una misma instrucción de maquina se aplica sobre una operación aritmética puntual la cual opera varios elementos de datos de forma paralela vs el modelo **escalar** tradicional, donde hasta el momento se empleaba una instrucción la cual solo podía procesar un único dato a la vez.

Por otra parte, en la arquitectura x86-64, el paradigma **SIMD**, se puede ver empleado en extensiones de conjuntos de instrucciones, como lo son: **SSE** (Streaming SIMD Extensions), **AVX** (Advanced Vector Extensions), **AVX2** (Advanced Vector Extensions 2), y **AVX-512** (Advanced Vector Extensions 512) ya que estas introducen registros de tipo vectorial, los cuales tienen un tamaño físico entre los 128 bits hasta los 512 bits y los mismos son capaces de guardar múltiples valores de tipo flotante o enteros y simultáneamente puede operar con todo el conjunto de datos mediante un set de instrucciones vectoriales como lo pueden ser **vmulps** o **vaddps**. De la misma manera, el presente proyecto indaga precisamente en la capacidad que tiene el hardware moderno para poder resolver un problema representativo del cómputo científico y estadístico denominado como la *normalización de un arreglo de datos*, la que a su vez requiere el calculo de la media, la varianza y los valores extremos del arreglo o conjunto de números y a partir de dicha información poder transformar cada elemento a una escala estandarizada. A su vez, este problema en particular, es un buen caso para poder ejecutarlo con el uso de un conjunto de instrucciones vectoriales, ya que su patrón de acceso a memoria que es de tipo regular y sus operaciones aritméticas que al mismo tiempo son perfectamente vectorizables, las cuales son sumas, restas, multiplicaciones, divisiones y comparaciones. Del mismo modo, este caso permite comparar de forma directa, visual y cuantitativa el costo de una implementación escalar vs a una vectorizada, la cual se empleara en el lenguaje de programación de bajo nivel **Assembly**, utilizando el ensamblador NASM para la plataforma específica de Linux x86-64.

Adicionalmente como lo señalan Hennessy y Patterson, dentro de la clasificación de Flynn para arquitecturas paralelas, en una máquina SIMD “*la misma instrucción es ejecutada por múltiples procesadores usando distintos flujos de datos*” [Patterson and Hennessy, 2019]¹ En el mismo sentido, Bryant y O’Hallaron, describen el origen del acrónimo y contrastan explícitamente el modelo vectorial con el escalar: “*SIMD (que se pronuncia “sim-di”) es el acrónimo de “instrucción única, datos múltiples” (single-instruction, multiple-data=.* Asimismo, las versiones más recientes de los procesadores x86-64 implementan las instrucciones multimedia AVX. Además de proporcionar un superconjunto de las instrucciones SSE, AVX amplía el tamaño de los registros vectoriales. Nuestra exposición se

¹Traducción propia. Cita original en inglés: “*the same instruction is executed by multiple processors using different data streams*”. Hennessy, John L.; Patterson, David A. *Computer Architecture: A Quantitative Approach*. 6.^a ed. Cambridge, MA: Morgan Kaufmann / Elsevier, 2019, p. 11.

basa en AVX2, la segunda versión de AVX, introducida con el procesador Core i7 Haswell en 2013. (...) Las instrucciones AVX2 permiten realizar operaciones vectoriales sobre estos registros, como sumar o multiplicar en paralelo ocho o cuatro conjuntos de valores. [Bryant and O'Hallaron, 2016]
”2

Finalmente con este marco teórico conceptual como base principal, el presente informe técnico tiene como propósito no solamente el poder implementar las dos versiones de un algoritmo de normalizador estadístico (escalar y vectorial), sino también el poder cuantificar en que medida la vectorización cumple con su objetivo teórico de poder acelerar el computo. Asimismo, se evaluarán las diversas limitaciones prácticas como lo es el manejo de datos no alineados, el remanente cuando N (tamaño del arreglo de datos) no es un múltiplo del ancho del registro vectorial, o el ancho de banda de memoria, el cual impide alcanzar el *speedup* teórico de $8\times$ que ofrece el conjunto de instrucciones AVX2 sobre un kernel puramente escalar de punto flotante de 32 bits.

1.1. Objetivos

Objetivo general

Implementar, verificar y comparar el rendimiento de un algoritmo normalizador estadísticos de arreglos de tipo flotante entre dos versiones: una escalar y una vectorizada (AVX2), escritas en ensamblador NASM para la arquitectura x86-64 sobre Linux, con la finalidad de poder cuantificar el impacto real del conjunto de instrucciones de vectorización SIMD frente al cómputo tradicional con un conjunto de instrucciones escalares.

Objetivos específicos

- Implementar en NASM, siguiendo la convención de llamada System V AMD64, una versión **escalar** de las rutinas `sum_array`, `compute_stats` y `normalize_array`, y verificar los datos obtenidos frente a una referencia calculada en Python.
- Implementar en NASM una versión **vectorizada** de las mismas rutinas utilizando el conjunto de instrucciones AVX2 (o SSE/AVX como alternativa si el procesador disponible no soporta AVX2), incluyendo una estrategia explícita de reducción horizontal y de manejo del remanente cuando el tamaño del arreglo no es múltiplo del ancho del registro vectorial.
- Diseñar y ejecutar un conjunto de casos de prueba que cubra los escenarios borde más relevantes como lo son: $N = 0$, $N = 1$, remanente no nulo, varianza cero, valores negativos y valores extremos, para de esta manera poder garantizar la solución numérica de ambas implementaciones (escalar y vectorial).
- Medir cuantitativamente el tiempo de ejecución, el *speedup* en su versión vectorial y métricas de bajo nivel (como lo son el IPC, fallos de caché mediante `perf stat`) para distintos tamaños de arreglo, e interpretar de forma puntual los resultados teniendo en cuenta la Ley de Amdahl y los límites de ancho de banda de memoria.

²Traducción propia. Cita original en inglés: “SIMD (pronounced “sim-dee”) is the acronym for “single-instruction, multiple-data.” More recent versions of x86-64 processors implement the AVX multimedia instructions. In addition to providing a superset of the SSE instructions, AVX expands the size of the vector registers. Our presentation is based on AVX2, the second version of AVX, introduced with the Core i7 Haswell processor in 2013. (...) AVX2 instructions can then perform vector operations on these registers, such as adding or multiplying eight or four sets of values in parallel.” Bryant, Randal E.; O'Hallaron, David R. *Achieving Greater Parallelism with SIMD Instructions* (CS:APP3e Web Aside OPT:SIMD), material complementario oficial del libro *Computer Systems: A Programmer's Perspective*, 3.^a ed. Pittsburgh: Carnegie Mellon University, 2016, p. 1.

- Verificar mediante GDB, a nivel de registro y de memoria, que la ejecución de la versión vectorizada corresponde exactamente al comportamiento que se espera en el kernel.

2. Descripción del entorno de pruebas

Es importante resaltar que con la finalidad de poder garantizar la trazabilidad y la reproducibilidad a futuro de los resultados obtenidos y presentados en este informe, a continuación se documenta de forma explícita el entorno de *hardware* y *software* en el cual se desarrolló, compiló y ejecutó tanto la versión con el conjunto de instrucciones de tipo escalar como la versión con el conjunto de instrucciones de tipo vectorial del algoritmo propuesto denominado “*normalizador estadístico*”. Asimismo esta sección es de suma importancia para el presente proyecto, puesto que el comportamiento de las instrucciones vectoriales (AVX2) dependen directamente del tipo de procesador utilizado, ya que el mismo de tener soporte a dicha extensión del conjunto de instrucciones (AVX2), debido a que sino el programa realizado fallaría a la hora de la ejecución con una excepción de instrucción ilegal.

2.1. Hardware

El presente proyecto se ejecutó sobre una máquina con Linux Mint, cuyo procesador es un AMD Ryzen 5 7520U de 4 núcleos. Asimismo al ejecutar el comando `lscpu` se confirma que este procesador sí soporta el conjunto de instrucciones AVX2, así como FMA (*Fused Multiply-Add*) y SSE4.2, por lo que **no fue necesario aplicar la ruta de contingencia a SSE/AVX** descrita en el enunciado del proyecto. Por lo antes comentado, la versión vectorial se implementó y se probó utilizando el conjunto de instrucciones AVX2 de forma directa.

```

1 Model name:      AMD Ryzen 5 7520U with Radeon Graphics
2 Architecture:    x86_64
3 CPU(s):          4
4 Vendor ID:       AuthenticAMD
5 Flags:           ... sse sse2 ssse3 sse4_1 sse4_2 avx avx2 fma f16c
6                  bmi1 bmi2 popcnt aes xsave rdrand rdseed adx
7                  clflushopt sha_ni ...

```

Listing 1: Salida de `lscpu` (modelo de CPU y Flags relevantes)

Por otra parte, los indicadores de mas importancia para este proyecto, son los resaltados que se logran observar en la lista completa de **Flags**, los cuales son: **avx**, **avx2**, **fma**, **sse4_1** y **sse4_2**. De la misma manera se verifica que los cinco están presentes, lo que a su vez confirma que este entorno es totalmente apto para ejecutar el kernel vectorizado con el conjunto de instrucciones AVX2 de 256 bits sin necesidad alguna de cambiarlo por instrucciones SSE.

2.2. Comparación de extensiones vectoriales: SSE vs. AVX2

Es importante mencionar que dado que la elección de la extensión vectorial condiciona directamente el diseño de la versión vectorizada del proyecto (Sección 4), puesto que mientras las instrucciones SSE (Streaming SIMD Extensions) estas fueron introducidas en 1999 por Intel con el procesador Pentium III utilizan registros de 128 bits los cuales les permiten procesar hasta 4 valores de 32 bits por ciclo de reloj, en comparación a las AVX2 (Advanced Vector Extensions 2), las cuales fueron lanzadas en 2013 con la arquitectura Haswell de Intel y utilizan registros que duplican el ancho de banda puesto que son de 256 bits, logrando de esta manera poder procesar ocho valores simultáneamente. Lo anterior se puede evidenciar en la Tabla 1, la cual resume las diferencias anteriormente mencionadas entre SSE y AVX2.

Criterio	SSE (Streaming SIMD Extensions)	AVX2 (Advanced Vector Extensions 2)
Tamaño de registro	128 bits (registros XMM)	256 bits (registros YMM)
Operaciones simultáneas (<i>float</i> de 32 bits)	Hasta 4 operaciones por ciclo	Hasta 8 operaciones por ciclo
Arquitectura de instrucciones	Destructiva (el resultado siempre sobrescribe uno de los operandos de entrada)	No destructiva (tiene una codificación VEX / FMA3): asimismo esta permite 3 operandos y a su vez permite guardar el resultado en un registro separado y no sobrescribir la información
Compatibilidad de hardware	Universal (prácticamente se encuentra en cualquier CPU x86 desde 1999)	CPUs desde 2013 en adelante (Arquitecturas Intel Haswell / AMD Bulldozer o posteriores)
Consumo eléctrico y térmico	Moderado y estable, puesto que no altera las frecuencias base del procesador	Elevado, ya que puede generar picos de calor y activar el <i>AVX offset</i> (reducción de la frecuencia de la CPU)
Impacto de la velocidad de RAM	Menor dependencia del ancho de banda de memoria	Alta dependencia, puesto que si la RAM es lenta en muchas ocasiones se genera un cuello de botella y AVX2 pierde eficacia
Casos de uso ideales	Aplicaciones generales, audio clásico y sistemas antiguos	Edición de video 4K/8K, renderizado 3D, IA y criptografía

Cuadro 1: Comparación entre las extensiones vectoriales SSE y AVX2.

Nota: La información para realizar la tabla anterior se obtuvo del *Intel® 64 and IA-32 Architectures Software Developer's Manual* [Intel Corporation, 2026].

2.3. Software

La Tabla 2 resume las diversas versiones de las herramientas de desarrollo, ensamblado, compilación y depuración utilizadas para la obtención de los distintos datos.

Herramienta	Versión
Sistema operativo	Linux Mint 22.3
NASM	2.16.01
GCC	13.3.0 (Ubuntu 13.3.0-6ubuntu2 24.04.1)
GDB	15.1 (Ubuntu 15.1-1ubuntu1 24.04.1)
perf	7.0.14
Python	3.12.3
NumPy	1.26.4

Cuadro 2: Versiones de las herramientas usadas en el entorno de pruebas.

3. Implementación escalar

En primer lugar es importante mencionar que la versión escalar del algoritmo denominado "normalizador estadístico", se encuentra en la ruta `asm/scalar/stats_scalar.asm` del repositorio en GitHub que se puede acceder con el siguiente link: <https://github.com/steven-loaiza-cr/Proyecto-Arquitectura>. A su vez, esta sección es clave para el desarrollo del presente proyecto, puesto que la versión escalar es fundamental ya que es contra esta que se pueden contrastar los diversos resultados, tanto de la exactitud numérica de los resultados obtenidos como el tiempo de ejecución que se obtiene en su contra parte la versión vectorizada, la cual esta descrita en la Sección 4.

En segundo lugar, es fundamental indicar que esta versión utiliza exclusivamente instrucciones de punto o tipo flotante **escalares** de la familia SSE, como lo pueden ser las siguientes: (`movss`, `addss`, `subss`, `mulss`, `divss`, `comiss` y `cvtsi2ss`), las cuales solamente operen sobre los 32 bits menos significativos de los registros XMM que fueron utilizados en la elaboración de la versión escalar. En otras palabras, aunque se sabe que los registros XMM tienen físicamente un ancho de banda de 128 bits de capacidad, el sufijo `ss` (*scalar single-precision*), en las diversas instrucciones anteriormente mencionadas hace que en cada una de estas instrucciones se procese únicamente un solo valor de tipo `float` de 32 bits por iteración, dejando de esta manera, los 96 bits restantes de los registros XMM sin utilizar. De la misma manera, esta decisión no fue accidental, puesto que ese desaprovechamiento deliberado del ancho de los registros XMM es lo que nos permite poder cuantificar más adelante la ganancia real que aporta la vectorización al presente proyecto.

En tercer lugar, las tres rutinas implementadas (`sum_array`, `compute_stats` y `normalize_array`) describen exactamente las mismas firmas que están declaradas en el archivo de cabecera denominado `include/stats.h`, por lo que el archivo `driver.c` se puede enlazar indistintivamente contra `obj/stats_scalar.o` u `obj/stats_vector.o` para de esta manera poder producir los ejecutables de `bin/norm_scalar` y `bin/norm_vector`. Esta manera, garantiza que la única variable entre las diversas mediciones sea el kernel de cómputo y no la infraestructura de entrada/salida ni la de temporización.

3.1. Convención de llamada y prólogo/epílogo escalar

En el presente proyecto todo el código de ensamblador respeta la convención de llamada **System V AMD64 ABI**, que es la que utiliza Linux x86-64 y por lo tanto, la que asume el compilador GCC para poder efectuar su proceso en el `driver.c`. Asimismo, bajo esta convención los argumentos de los enteros y los diversos punteros, se deben pasar en forma ordenada y por los registros `rdi`, `rsi`, `rdx`, `rcx`, `r8` y `r9`, mientras que los argumentos de los punto o tipo flotantes se deben pasar por una secuencia independiente de los registros `xmm0`–`xmm7`. De la misma manera, esto es fundamental y explica la asignación de los distintos registros de `normalize_array`, puesto que `mean` y `stddev` son respectivamente el cuarto y el quinto argumento de dicha función, pero estos argumentos no llegan por medio de `rcx` y `r8`, sino en `xmm0` y `xmm1`, puesto que el ABI asigna los registros *por clase de argumento* y no por posición absoluta en la lista. De forma similar, el valor de retorno de tipo `float` se devuelve por medio del registro `xmm0`, debido a que por la cual la rutina `sum_array` puede terminar con un simple `ret` sin ninguna instrucción adicional de movimiento.

Por otra parte, el ABI clasifica los registros de propósito general en dos diversos grupos. En primer lugar, los registros *caller-saved* (`rax`, `rcx`, `rdx`, `rsi`, `rdi`, `r8`–`r11`), los cuales pueden ser destruidos libremente por la función llamada, en contraste a los registros denominados *callee-saved* (`rbx`, `rbp`, `r12`–`r15`), los cuales deben devolverse de forma intacta a quien invocó la rutina. Por esta razón, es importante saber que la única rutina que necesita un prólogo y epílogo explícitos es `compute_stats`, puesto que es la única rutina que emplea registros *callee-saved* para almacenar los

cuatro punteros de salida, los cuales se observan a continuación:

```

1 compute_stats:
2     push    rbx                ; prologo: se preservan los registros
3     push    r12               ; callee-saved que exige la ABI System V
4     push    r13
5     push    r14
6     push    r15
7
8     mov     r12, rdx           ; mean_ptr (3er argumento)
9     mov     r13, rcx           ; var_ptr  (4to argumento)
10    mov     r14, r8            ; min_ptr  (5to argumento)
11    mov     r15, r9            ; max_ptr  (6to argumento)
12
13    ; ... cuerpo de la rutina ...
14
15 .cs_retorno:
16    pop     r15                ; epilogo: se restauran en ORDEN INVERSO
17    pop     r14                ; (esto es de suma importancia)
18    pop     r13                ; al de los push, para desenrollar correctamente la pila
19    pop     r12
20    pop     rbx
21    ret

```

Listing 2: Prólogo y epílogo de `compute_stats`: preservación de los registros *callee-saved*

De la misma manera, la justificación de este diseño, se baso en los cuatros punteros de salida los cuales llegan en `rdx`, `rcx`, `r8` y `r9`, que son todos registros denominados *callee-saved*. Asimismo, es importante indicar que si el cuerpo de la rutina de `compute_stats` llegara a utilizar una subfunción, o simplemente si se reutilizaran esos registros como registros temporales dentro de los bucles, las direcciones de escritura se perderían y el programa terminaría escribiendo los resultados estadísticos obtenidos en posiciones erróneas en memoria. Por esta razón, copiarlos de forma inmediata a `r12–r15` garantiza que las direcciones sobrevivan sin importar el código que se efectuó dentro de la rutina de `compute_stats`. Además el único costo extra es un único `push` y un único `pop` por cada registro, cuyo costo es realmente despreciable frente al del bucle principal, puesto que solo es una vez por llamada y no por elemento del arreglo. Asimismo, el registro `rbx` se preserva para mantener la simetría y mantenibilidad del código, aunque en la versión escalar en ningún momento se llegue a utilizar, es mas seguro preservarlo mientras que no hacerlo llega a constituir una violación directa del ABI.

A su vez, es importante destacar que en ninguna de las 3 rutinas escalares, se reservo un marco de pila (`sub rsp, N`) ni utiliza `rbp` como un puntero para el marco. Lo anterior se debe a que todas las variables intermedias utilizadas (índice, acumuladores, mínimo, máximo y media) caben perfectamente en los registros. De la misma manera, el mantener todas estas variables sobre los registros evitan accesos innecesarios a la pila dentro del bucle principal y deja a `sum_array` y `normalize_array` como funciones hoja (*leaf functions*), lo que simplifica el código como su futura inspección en GDB.

Rutina	Registro	Uso
sum_array	rdi	Puntero base del arreglo de entrada
	esi	n (cantidad de elementos)
	eax	Índice i del recorrido
	xmm0	Acumulador de la suma y valor de retorno
	xmm1	Temporal: <code>arr[i]</code>
compute_stats	rdi, esi	Arreglo y n
	r12-r15	Copias de <code>mean*</code> , <code>var*</code> , <code>min*</code> , <code>max*</code>
	eax	Índice i (se reinicia entre pasadas)
	xmm0	Acumulador de la suma y, luego, la media
	xmm1, xmm2	Mínimo y máximo corrientes
	xmm3	Temporal: <code>arr[i]</code> y $(x - \mu)^2$
	xmm4	n convertido a <code>float</code> (divisor)
	xmm5	Acumulador de $\sum (x - \mu)^2$
normalize_array	rdi, rsi	Punteros de entrada y de salida
	edx	n (cantidad de elementos)
	eax	Índice i
	xmm0, xmm1	Media y desviación estándar (argumentos)
	xmm2	Constante 0,0 para la comparación
	xmm3	Temporal del elemento en proceso

Cuadro 3: Asignación de registros en la implementación escalar.

3.2. sum_array

La rutina `sum_array` calcula $\sum_{i=0}^{n-1} \text{arr}[i]$ y genera el patrón básico del recorrido que reutilizan las demás rutinas, ya que esta rutina inicializa un índice y un acumulador, compara la condición de salida al inicio del bucle, carga los diversos elementos de arreglo de datos, realiza operaciones aritméticas e incrementa el índice en cada pasada del bucle.

```

1 sum_array:
2     xor     eax, eax           ; i = 0 (xor de un registro consigo mismo)
3     xorps   xmm0, xmm0        ; acumulador = 0.0 (limpia los 128 bits)
4
5 .sum_loop:
6     cmp     eax, esi           ; compara i contra n
7     jge     .sum_done          ; si i >= n, el arreglo ya se recorrio
8     movss   xmm1, [rdi + rax*4]; xmm1 = arr[i]; base + i * 4 (4 bytes/float)
9     addss   xmm0, xmm1        ; acumulador += arr[i]
10    inc     eax                ; i++
11    jmp     .sum_loop
12
13 .sum_done:
14    ret                         ; el resultado ya esta en xmm0 (ABI)

```

Listing 3: Fragmento clave de `sum_array` (versión escalar)

Es importante resaltar que la acumulación se realiza directamente sobre `xmm0` con `addss` precisamente puesto que según la ABI (como anteriormente se menciona), `xmm0` es el registro de retorno para valores de tipo `float` y a su vez, genera que el acumular en cualquier otro registro obligaría a

añadir una instrucción de `movss` final hacia `xmm0` antes del `ret`.

3.3. compute_stats

La rutina `compute_stats` calcula la media, la varianza poblacional, el mínimo y el máximo del arreglo, y escribe los cuatro resultados obtenidos a través de los punteros recibidos como argumento. A su vez, es importante indicar que la varianza se define como:

$$\sigma^2 = \frac{1}{n} \sum_{i=0}^{n-1} (x_i - \mu)^2, \quad \mu = \frac{1}{n} \sum_{i=0}^{n-1} x_i, \quad (1)$$

y esta fórmula coloca una restricción algorítmica, puesto que la media debe conocerse por completo antes de poder acumular las diferencias al cuadrado. Por esta razón, la rutina se estructura en **dos pasadas** sobre el arreglo.

La primer pasada tiene como objetivo el poder resolver simultáneamente las 3 reducciones (suma, mínimo y máximo), puesto que las tres son independientes entre sí y comparten un mismo recorrido y por lo tanto, también comparten los mismos accesos a memoria. De la misma manera, el poder realizarlas en un mismo bucle evita dos recorridos adicionales del arreglo, lo que resulta muy eficiente y relevante para cuando se tienen arreglos de datos muy grandes (valores grandes de n), donde el costo principal no es el costo de la operación aritmética sino el de ir trayendo los diversos datos desde la memoria principal, esto se debe a la velocidad de dicha memoria.

```

1  test     esi, esi           ; Compuerta AND interna: solo actualiza banderas
2  jle     .cs_arreglo_vacio   ; caso borde: n <= 0
3
4  xorps    xmm0, xmm0         ; sum = 0.0
5  movss    xmm1, [rdi]        ; min = arr[0]
6  movss    xmm2, [rdi]        ; max = arr[0]
7  xor      eax, eax           ; i = 0
8
9  .cs_bucle_suma_min_max:
10  cmp     eax, esi            ;
11  jge     .cs_fin_suma_min_max
12  movss    xmm3, [rdi + rax*4] ; x = arr[i]
13  addss    xmm0, xmm3         ; sum += x
14  comiss   xmm3, xmm1         ; compara x contra el minimo actual
15  jae     .cs_verificar_max    ; si x >= min, no hay nuevo minimo
16  movss    xmm1, xmm3         ; min = x
17
18  .cs_verificar_max:
19  comiss   xmm3, xmm2         ; compara x contra el maximo actual
20  jbe     .cs_siguiete_elemento ; si x <= max, no hay nuevo maximo
21  movss    xmm2, xmm3         ; max = x
22
23  .cs_siguiete_elemento:
24  inc     eax
25  jmp     .cs_bucle_suma_min_max
26
27  .cs_fin_suma_min_max:
28  cvtsi2ss xmm4, esi          ; convierte n (entero) a float
29  divss    xmm0, xmm4         ; mean = sum / n

```

Listing 4: Primera pasada de `compute_stats`: suma, mínimo y máximo en solo bucle

A partir de esta sección de código de la rutina `compute_stats`, es importante justificar diversas decisiones de diseño. En primer lugar, la verificación del caso borde se realiza con `test esi, esi` seguido de `jle`, en lugar de un `cmp esi, 0`, puesto que la instrucción `test` ejecuta un AND interna del registro consigo mismo y actualiza de esta manera las banderas sin necesidad de escribir ningún resultado, por lo que se puede afirmar que es un byte más corta que la comparación con un inmediato

y produce exactamente la misma respuesta de información de signo y de cero. A su vez, el uso de `jle` en vez de `je` cubre además el caso de un n negativo, ya que aunque el `driver.c` ya descarta el leer un archivo de entrada con n negativo, pero aunque se verifique anteriormente si conviene verificarlo nuevamente en el kernel.

Por otra parte, la necesidad de verificar explícitamente $n \leq 0$ en esta rutina (`compute_stats`), en comparación de la rutina (`sum_array`), se debe a 2 motivos: en primer lugar es que el mínimo y el máximo requieren inicializarlos, pero el inicializarlos en 0,0 sería incorrecto para un arreglo de puros valores negativos, puesto que el máximo reportado sería de 0,0 y este valor que no pertenece al arreglo de un caso en específico que tenga puros valores negativos, por lo se inicializan en `arr[0]`, lectura que solamente se puede realizar si existe por lo menos un elemento en el arreglo de datos. Ahora bien, en segundo lugar, se tiene el motivo del cálculo de la media el cual implica una división entre n ; con $n = 0$ se produciría una división entre cero la cual generaría un error y se propagaría al resto de variables como la varianza y a la rutina del arreglo normalizado.

De forma similar, para el mínimo y el máximo se optó por la utilización de la instrucción `comiss` con saltos condicionales en lugar de `minss/maxss`, esto se debió por legibilidad y de trazabilidad, ya que facilita su posterior inspección paso a paso en GDB. Asimismo, es importante indicar que el costo de esta decisión es la presencia de dos saltos condicionales dependientes de los datos dentro del bucle principal. Ahora bien, esta diferencia es relevante para la comparación posterior, ya que la versión vectorizada del algoritmo sí emplea `vmnps/vmaxps`, es decir, una estrategia sin saltos condicionales, por lo que contribuye al *speedup*.

Finalmente, la conversión del n se realiza con el uso de la instrucción `cvtsi2ss xmm4, esi`, la cual traduce el entero de 32 bits a su representación en punto flotante de precisión simple. Además, este valor se conserva en el registro `xmm4`

```

1  xorps    xmm5, xmm5          ; sum = 0.0
2  xor     eax, eax             ; i = 0 (se reinicia el indice)
3
4  .cs_bucle_varianza:
5      cmp     eax, esi
6      jge     .cs_fin_varianza
7      movss   xmm3, [rdi + rax*4] ; x = arr[i]
8      subss   xmm3, xmm0        ; x - mean (mean quedo en xmm0)
9      mulss   xmm3, xmm3        ; (x - mean)^2
10     addss   xmm5, xmm3        ; sum += (x - mean)^2
11     inc     eax
12     jmp     .cs_bucle_varianza
13
14 .cs_fin_varianza:
15     divss   xmm5, xmm4        ; var = sum / n
16     movss   [r12], xmm0      ; *mean
17     movss   [r13], xmm5      ; *var
18     movss   [r14], xmm1      ; *min
19     movss   [r15], xmm2      ; *max
20     jmp     .cs_retorno
21
22 .cs_arreglo_vacio:
23     xorps   xmm0, xmm0        ; caso borde n <= 0: los cuatro
24     movss   [r12], xmm0      ; resultados se fijan en 0.0, tal
25     movss   [r13], xmm0      ; como lo exige el contrato de
26     movss   [r14], xmm0      ; stats.h y el enunciado del proyecto,
27     movss   [r15], xmm0      ; evitando la divisionentre cero

```

Listing 5: Segunda pasada de `compute_stats`: varianza poblacional y escritura de resultados

La segunda pasada se observa que el cuadrado de la diferencia se obtiene con la instrucción `mulss xmm3, xmm3`, esta ejecuta una multiplicación del registro por sí mismo en lugar de utilizar una rutina de potencia. Asimismo, se emplea un acumulador distinto el cual es el registro (`xmm5`), en comparación con la primera pasada, que utilizaba el registro `xmm0`, puesto que este registro se debe

conservar de forma intacta la media calculada previamente durante todo el recorrido, de la segunda pasada.

Otro punto de interés es la rama del código de `.cs_arreglo_vacio`, la cual no termina con un `jmp`, sino que cae por caída natural sobre la siguiente etiqueta `.cs_retorno`, puesto que esta etiqueta contiene el epílogo mostrado en el Listado 2. Esto se efectuó de esta manera para evitar duplicar la secuencia de restauración de los diversos registros y garantiza de esta manera que exista un único punto de salida de la rutina, lo que a su vez asegura que los registros denominados callee-saved se puedan restauren por igual en ambos caminos de ejecución.

Por último, también es fundamental mencionar que aunque el enunciado del proyecto sugería reutilizar `sum_array` mediante el uso de la instrucción `call` dentro de rutina `compute_stats`, en la versión escalar se optó por replicar el bucle de suma dentro de la primera pasada. La justificación teórica conlleva dos aspectos, en primer lugar el desarrollo de esta sección resultó de la reutilización del código de rutina de `sum_array`, puesto que durante la codificación o el desarrollo del mismo, se reconoció el error ejecutado, ya que la implementación de la estructura necesaria ya se encontraba integrada en la rutina anterior, por consecuente se decidió terminar con la codificación y no utilizar el uso de la instrucción `call` y en segundo lugar, cuando se efectuó el análisis teórico del algoritmo se visualizó que al desarrollarse la codificación permitió evitar un recorrido adicional del arreglo, puesto que el mismo bucle efectuaba el cálculo del mínimo y del máximo en forma paralela en comparación si se hubiera utilizado la instrucción `call` para llamar a `sum_array`, esta lógica habría obligado a recorrer los datos una vez más. Ahora bien, la versión vectorizada, en cambio, sí efectúa esta llamada, lo que a su vez constituye una diferencia de diseño entre ambas implementaciones que debe tenerse presente al interpretar los distintos tiempos obtenidos en la Sección 6.

3.4. `normalize_array`

La rutina `normalize_array` aplica la transformación de estandarización sobre el arreglo de datos iniciales, teniendo en cuenta la siguiente fórmula:

$$\text{out}[i] = \frac{\text{in}[i] - \mu}{\sigma}, \quad (2)$$

de manera que el arreglo resultante (arreglo estandarizado) va a poseer una media de cero y una desviación estándar de 1. Asimismo, es importante indicar que esta rutina en específico no calcula σ , que es la desviación estándar puesto que obtiene esta información directamente del `driver.c` que aplica `sqrtdf` sobre la varianza que resultó por la rutina `compute_stats`, y finalmente la entrega ya lista en `xmm1`.

```

1 normalize_array:
2   xorps    xmm2, xmm2           ; xmm2 = 0.0
3   comiss   xmm1, xmm2           ; compara stddev contra 0.0
4   je       .na_copiar_directo    ; si stddev == 0 -> copia sin dividir
5   xor      eax, eax             ; i = 0
6
7   .na_bucle_normalizacion:
8   cmp      eax, edx             ; edx = n (3er argumento entero)
9   jge      .na_fin
10  movss     xmm3, [rdi + rax*4]    ; x = in[i]
11  subss     xmm3, xmm0            ; x - mean
12  divss     xmm3, xmm1            ; (x - mean) / stddev
13  movss     [rsi + rax*4], xmm3    ; out[i] = resultado
14  inc      eax
15  jmp      .na_bucle_normalizacion
16
17 .na_copiar_directo:             ; caso borde: sigma = 0
18  xor      eax, eax

```

```

19
20 .na_bucle_copia:
21     cmp     eax, edx
22     jge     .na_fin
23     movss   xmm3, [rdi + rax*4]      ; out[i] = in[i] tal cual, sin
24     movss   [rsi + rax*4], xmm3      ; dividir entre cero
25     inc     eax
26     jmp     .na_bucle_copia
27
28 .na_fin:
29     ret

```

Listing 6: Fragmento clave de `normalize_array` (versión escalar)

En esta rutina la decisión de diseño mas importante fue el tratamiento del caso $\sigma = 0$, ya que este caso en particular ocurre siempre que todos los elementos del arreglo son iguales. Asimismo, matemáticamente se observa que la ecuación (2) queda indeterminada en ese escenario en específico. Por esta razón, la verificación se realiza una única vez antes de entrar a cualquier bucle y no en cada iteración, puesto que dado que `stddev` es un argumento el cual no va a cambiar durante toda la ejecución de esta rutina en específico, por esa razón el comprobarlo dentro del bucle principal añadiría n comparaciones y n saltos sin aportar ninguna información nueva al algoritmo planteado. De la misma manera, el resultado es un diseño con dos caminos de ejecución mutuamente excluyentes, puesto que cada uno tiene su propio bucle, en primer lugar el camino normal de normalización y en segundo lugar el camino de la ejecución de copia directa que exige el contrato de `stats.h`.

Por otra parte, el caso $n = 0$ se resuelve nuevamente sin necesidad alguna de mas código adicional, debido a que la condición de salida se evalúa antes del cuerpo del bucle principal, esto se debe a que ninguno de los dos bucles llega a ejecutarse y la rutina retorna sin haber antes escrito en `out`, lo cual a su vez es el comportamiento deseado dado que no existe ningún elemento que normalizar.

Por último, es importante anticipar un aspecto fundamental que es sumamente determinante en el análisis de rendimiento que se ejecutara posteriormente en la Sección 6. Esto se debe a que la instrucción `divss` es con diferencia, la operación de punto flotante más costosa de nuestro kernel, ya que esta tiene una latencia del orden de 10 a 15 ciclos y un *throughput* muy inferior al de la suma o la multiplicación. Esto se observa dentro del bucle, ya que se esta instrucción (`divss`) se ejecuta n veces. Por otra parte, una optimización inmediata consiste en calcular una sola vez el recíproco de $1/\sigma$ y sustituir las n divisiones por n multiplicaciones. De la misma manera, es importante indicar que esta instrucción se conservó deliberadamente en la versión escalar la funciona como la versión referencia para de esta manera poder mantener la correspondencia literal con la ecuación (2) y con la versión vectorizada, la cual también va a utilizar `vdivps` por la misma razón, de modo que la comparación entre ambas mida exclusivamente el efecto de la vectorización y no el de posible mejor optimización algebraica aplicada solo a una de las dos versiones del mismo algoritmo.

4. Implementación vectorial

La versión vectorizada en el algoritmo denominado "normalizador estadístico" se encuentra en la ruta `asm/vector/stats_vector.asm` del repositorio en GitHub que se puede acceder con el siguiente link: <https://github.com/steven-loaiza-cr/Proyecto-Arquitectura>. Asimismo esta versión expone exactamente las mismas 3 rutinas declaradas en `include/stats.h` que la versión escalar descrita en la Sección 3. Esto tambien se logra observar en la arquitectura del proyecto, puesto que el archivo `Makefile` produce dos ejecutables, los cuales son en primer lugar el `(bin/norm_scalar)` y en segundo lugar el `(bin/norm_vector)`, estos a su vez comparten el mismo archivo tipo objeto `(obj/driver.o)` y difieren únicamente en el objeto del kernel con el que cada uno de estos se enlaza. Lo anterior es de suma importancia, porque refleja que cualquier diferencia de tiempo obtenida y presentada en la

Sección 6 es atribuida directamente y exclusivamente al cómputo en ensamblador y no a causa de diversos factores como lo son: la infraestructura de entrada/salida, de temporización o de impresión de resultados.

El pilar fundamental de operación de esta versión consiste en sustituir las instrucciones escalares de sufijo **ss** (*scalar single*) por sus equivalentes empaquetadas de sufijo **ps** (*packed single*), las cuales operan paralelamente sobre los 8 valores del dato tipo **float** de 32 bits que caben en un registro YMM, el cual tiene un ancho de banda de 256 bits (esto se evidencia en la Tabla 1).

4.1. Justificación de la extensión vectorial elegida

La elección de la extensión vectorial se tomó a partir de la verificación de las capacidades del procesador disponible y la cual esta documentada y disponible en el Listado 1 de la Sección 1. Asimismo, en esa tabla se confirma que el procesador es un AMD Ryzen 5 7520U el cual se va a utilizar para el entorno de pruebas y el mismo expone los siguientes indicadores: **avx**, **avx2**, **fma**, **sse4_1** y **sse4_2**, por lo cual demuestra que la ruta de emergencia hacia el uso de SSE/AVX prevista en el enunciado no fue necesaria para este proyecto en particular y se implementó directamente sobre **AVX2**. Las 3 alternativas consideradas y los diversos criterios que llevaron a descartar dos de ellas se presentan a continuación:

1. **SSE (registros XMM de 128 bits).**: esta elección fue descartada porque procesa únicamente cuatro valores de tipo **float** por cada instrucción ejecutada o procesada, lo que a su vez acota el *speedup* teórico máximo a $4\times$ en lugar de $8\times$. Asimismo, es importante indicar que el conjunto SSE emplea una codificación de instrucciones *destructiva* de dos operandos, en el cual el resultado sobrescribe obligatoriamente uno de los operandos de entrada (caso de la versión escalar). Lo anterior mencionado, obliga a insertar más instrucciones adicionales para copiar los datos (**movaps**), en cada ocasión que un operando se debe preservar o mantener para la ejecución de una operación posterior, penalizando de esta manera sobre el bucle principal del algoritmo implementado. Finalmente, su única ventaja se refleja en la compatibilidad universal con cualquier CPU x86 desde 1999, y que al mismo tiempo se considera totalmente irrelevante para el contexto del presente proyecto, puesto que el hardware de ejecución es conocido y esta documentado previamente.
2. **AVX-512 (registros ZMM de 512 bits).**: esta elección fue descartada por una razón totalmente material y física, puesto que el procesador del entorno de pruebas no la soporta, esto se logra visualizar en la ausencia de cualquier indicador que corresponde a la familia o grupo **avx512** en la salida de **lscpu**. De forma similar, el intentar ejecutar una instrucción que conlleva los registros ZMM en este hardware en específico provocaría una excepción de instrucción ilegal (**SIGILL**) y a su vez el cierre abrupto del programa. Asimismo, es importante indicar que AVX-512 no garantiza automáticamente el doble de rendimiento en comparación a AVX2, esto es causado debido a que muchas microarquitecturas generan una reducción de la frecuencia de reloj (*AVX offset*), la cual a su vez puede llegar a anular cierto porcentaje de la ganancia teórica en las distintas cargas de trabajo las cuales también están limitadas por el tipo memoria.
3. **AVX2 (registros YMM de 256 bits).**: esta elección fue la escogida, puesto que AVX2 duplica el ancho de banda en comparación con SSE, ya que procesa hasta 8 datos simultáneamente de tipo **float** por instrucción ejecutada y a su vez, utiliza la codificación VEX de tres operandos y la cual es de tipo no destructiva, ya que permite escribir el resultado en un registro distinto de los dos operandos de entrada.

Por otra parte, es importante resaltar que la disponibilidad del indicador `fma` permitió el poder emplear la instrucción `vmadd231ps` para de esta manera poder acumular las diferencias al cuadrado en una sola operación que contiene la formula de la varianza.

4.2. Convención de llamada y prólogo/epílogo vectorial

`compute_stats` respeta la convención de llamada System V AMD64 ABI: los seis argumentos de tipo puntero/entero (`arr`, `n`, `mean*`, `var*`, `min*`, `max*`) llegan en `rdi`, `esi`, `rdx`, `rcx`, `r8` y `r9`, en ese orden, y la función no retorna valor por registro, es decir, los resultados se escriben en memoria a través de los punteros recibidos.

El prólogo guarda en la pila los seis registros callee-saved que la rutina va a utilizar (`rbp`, `rbx`, `r12–r15`), cumpliendo con la obligación de la ABI de preservar su valor para el llamador. Inmediatamente después, copia los seis argumentos de entrada desde los registros caller-saved (`rdi`, `esi`, `rdx`, `rcx`, `r8`, `r9`) hacia los registros recién guardados. Esta relocalización es necesaria porque el cuerpo de la función invoca internamente a `sum_array` para calcular la media, y dicha llamada destruye el contenido de los registros caller-saved; al conservar los argumentos en registros callee-saved, se garantiza que sigan disponibles después del `call` sin depender de que `sum_array` los preserve.

Como consecuencia de esta llamada interna, el prólogo también debe restablecer la alineación de pila exigida por la ABI (16 bytes al entrar a cualquier función). Al ingresar a `compute_stats`, `rsp` está desplazado 8 bytes respecto a un múltiplo de 16 (por la dirección de retorno empujada por el `call` del llamador); los seis `push` del prólogo suman 48 bytes (múltiplo de 16) y no corrigen ese desplazamiento. Por ello, inmediatamente antes de llamar a `sum_array` se ejecuta `sub rsp, 8` para alinear la pila a 16 bytes, y tras el retorno se ejecuta `add rsp, 8` para devolver la pila a su estado previo. (ver en Sección 4.4).

El epílogo, en la etiqueta `.cs_cierre`, comienza con `vzeroupper` para poner en cero la mitad alta de todos los registros YMM, evitando la penalización de transición entre código AVX y código SSE que podría ejecutarse a continuación (por ejemplo, en el llamador o en otra rutina). Luego restaura los seis registros callee-saved con `pop`, en el orden exactamente inverso al usado en el prólogo, condición necesaria para que cada `pop` recupere el valor correcto de la pila tipo LIFO. Finalmente, `ret` retorna el control al llamador; dado que `compute_stats` no devuelve valor escalar, no se escribe nada en `xmm0` antes del retorno.

```

1 compute_stats:
2     push    rbp
3     push    rbx
4     push    r12
5     push    r13
6     push    r14
7     push    r15
8
9     mov     r12, rdi          ; arr
10    mov     r13d, esi         ; n
11    mov     rbx, rdx          ; mean*
12    mov     rbp, rcx          ; var*
13    mov     r14, r8           ; min*
14    mov     r15, r9           ; max*
15
16    ; ... cuerpo de la rutina ...
17
18 .cs_cierre:
19     vzeroupper                ; evita penalizacion de transicion AVX/SSE
20     pop     r15                ; pop de los registros callee-saved en orden inverso a como
21     pop     r14                ; se introdujeron a la pila para evitar segfaults.
22     pop     r13
23     pop     r12

```

```

24  pop     rbx
25  pop     rbp
26  ret                                ; Fin de la funcion

```

Listing 7: Prólogo y epílogo de `compute_stats` vectorial: preservación de los registros *callee-saved*

Rutina	Registro	Uso
sum_array	rdi, esi eax ymm0/xmm0 ymm1/xmm1	arr y n Índice i Acumulador vectorial/escalar de la suma; retorno Temporal: arr[i]
compute_stats	rdi,esi,rdx,rcx,r8,r9 r12-r15, rbx, rbp eax / ecx xmm0 xmm1 ymm2 ymm3/xmm3 ymm4/xmm4 ymm5/xmm5 ymm6, ymm7 xmm8 xmm9, xmm10	Argumentos de entrada: arr, n, mean*, var*, min*, max* Copias callee-saved de arr, n, min*, max*, mean*, var* Índice i (bucle y cola) / límite vectorial ($n \sim 7$) Suma $\rightarrow$ media μ ; reutilizado en la cola n convertido a float μ replicada en 8 carriles Acumulador de $\sum (x - \mu)^2 \rightarrow \sigma^2$ Acumulador de mínimo (semilla arr[0]) $\rightarrow$ mínimo final Acumulador de máximo (semilla arr[0]) $\rightarrow$ máximo final Temporales vectoriales: $x - \mu$ y arr[i] Temporal de reducción horizontal (reutilizado) Temporales de cola: arr[i] y $(x - \mu)^2$
normalize_array	rdi, rsi, edx eax / ecx xmm0/ymm2 xmm1/ymm3 xmm4 ymm5/xmm6	in, out, n Índice i (4 bucles) / límite vectorial Media (arg.); replicada y reutilizada en la cola Desviación estándar (arg.); replicada y reutilizada en la cola Constante 0,0 (comiss contra stddev) Temporal vectorial/escalar: carga y resultado (o copia si $\sigma = 0$)

Cuadro 4: Asignación de registros en la implementación vectorial.

4.3. sum_array vectorizado

La rutina `sum_array` vectorizada establece un patrón para las demás rutinas del proyecto y la misma consta de 4 etapas bien diferenciadas, las cuales son: cálculo del límite del bucle vectorial, bucle principal de ocho en ocho, reducción horizontal y finalmente un bucle escalar de cierre.

```

1  sum_array:
2      xor     eax, eax                ; i = 0
3      vxorps  ymm0, ymm0, ymm0       ; acumulador vectorial: 8 carriles a 0.0
4      mov     ecx, esi                ; ecx = n
5      and     ecx, ~7                 ; ecx = n redondeado abajo a multiplo de 8
6      test    ecx, ecx
7      jle     .sum_reduce             ; si n < 8, no hay bucle vectorial
8
9  .sum_vec_loop:
10     cmp     eax, ecx
11     jge     .sum_reduce             ; si i >= n, salta a reduccion horizontal
12     vmovups ymm1, [rdi + rax*4]     ; carga 8 floats de una sola vez
13     vaddps  ymm0, ymm0, ymm1       ; acumula carril por carril (8 sumas)
14     add     eax, 8                   ; i += 8 (no i++)
15     jmp     .sum_vec_loop
16
17 .sum_reduce:
18     ; --- reduccion horizontal: 8 carriles -> 1 escalar ---

```

```

19  vextractf128 xmm2, ymm0, 1      ; xmm2 = mitad alta (carriles 4-7)
20  vaddps  xmm0, xmm0, xmm2      ; 4 sumas parciales
21  vhaddps xmm0, xmm0, xmm0      ; suma horizontal dentro de 128 bits
22  vhaddps xmm0, xmm0, xmm0      ; xmm0[0] = suma total de los 8 carriles
23
24  .sum_scalar_tail:
25  cmp     eax, esi              ; elementos sobrantes (n % 8)
26  jge     .sum_done
27  vmovss  xmm1, [rdi + rax*4]
28  vaddss  xmm0, xmm0, xmm1
29  inc     eax
30  jmp     .sum_scalar_tail
31
32  .sum_done:
33  vzeroupper                    ; evita penalizacion de transicion AVX/SSE
34  ret

```

Listing 8: Fragmento clave de `sum_array` (versión vectorial AVX2)

Por otra parte, es importante. indicar que esta rutina fue proporcionada como un ejemplo completo por el profesor en el esqueleto del presente proyecto, esto con el fin de servir como un patrón base para que los distintos estudiantes replicaran esta misma estructura en las rutinas `compute_stats` y en `normalize_array`, las cuales se presentan en las Secciones 4.4 y 4.5. A continuación, se describe el funcionamiento de la misma.

El registro `ecx`, inicializado como copia de `n`, se redondea hacia abajo al múltiplo de 8 más cercano con `and ecx, ~7`, obteniendo así el límite del bucle vectorial. La comparación `test ecx, ecx / jle .sum_reduce` evalúa si ese límite es cero; como `ecx` vale `n` redondeado hacia el múltiplo más bajo de 8, esto equivale a $n < 8$ (con $n = 0$ como caso extremo, no como rama separada), y de cumplirse se salta directo a la reducción horizontal dejando todo el trabajo al bucle escalar de cierre. Caso contrario, se entra a `.sum_vec_loop`: mientras `eax` sea menor a `ecx`, `vmovups` carga 8 floats desde `arr[i]` en `ymm1`, `vaddps` los acumula carril por carril en `ymm0`, y `eax` avanza de 8 en 8, produciendo al final 8 sumas parciales independientes en lugar de un total único.

La reducción horizontal colapsa esos 8 carriles a un escalar: `vextractf128` extrae la mitad alta de `ymm0` a `xmm2`, `vaddps` la suma contra la mitad baja (4 sumas parciales), y dos `vhaddps` sucesivos reducen esos 4 valores a 1 ($4 \rightarrow 2 \rightarrow 1$), dejando el total en el elemento menos significativo de `xmm0`.

El remanente se procesa en `.sum_scalar_tail`: aquí la comparación `cmp eax, esi` es contra `n` original, y mientras `eax` sea menor a `n`, `vmovss/vaddss` suman un float a la vez sobre el acumulador ya reducido. Finalmente `vzeroupper` limpia la mitad alta de los registros YMM antes del `ret`, evitando la penalización de transición AVX/SSE.

4.4. `compute_stats` vectorizado

Esta rutina en particular, se considera la mas compleja de la implementación de todo el proyecto, puesto que la misma debe sostener simultáneamente los 3 acumuladores vectoriales de forma independiente (suma de cuadrados, el mínimo y el máximo). Asimismo, a diferencia con su contraparte escalar, sí reutiliza `sum_array` mediante una llamada explícita para obtener la media.

```

1  compute_stats:
2  push    rbp                  ; se preservan 6 registros callee-saved
3  push    rbx                  ; (48 bytes en la pila)
4  push    r12
5  push    r13
6  push    r14
7  push    r15
8
9  mov     r12, rdi              ; arr
10 mov     r13d, esi             ; n

```

```

11  mov     rbx, rdx                ; mean*
12  mov     rbp, rcx                ; var*
13  mov     r14, r8                 ; min*
14  mov     r15, r9                 ; max*
15
16  test    r13d, r13d
17  jle     .cs_vaciar_registros    ; caso borde n <= 0
18
19  ; --- mean = sum_array(arr, n) / n ---
20  mov     rdi, r12
21  mov     esi, r13d
22  sub     rsp, 8                  ; realinea rsp a 16 bytes antes del call
23  call    sum_array
24  add     rsp, 8                  ; deshace el ajuste
25  cvtsi2ss xmm1, r13d
26  divss   xmm0, xmm1             ; xmm0 = mean
27  movss   [rbx], xmm0            ; *mean
28
29  vbroadcastss ymm2, xmm0        ; mean replicada en los 8 carriles
30  vbroadcastss ymm4, dword [r12] ; semilla min = arr[0] en 8 carriles
31  vbroadcastss ymm5, dword [r12] ; semilla max = arr[0] en 8 carriles
32  vxorps   ymm3, ymm3, ymm3      ; acumulador sumsq = 0
33
34  xor     eax, eax
35  mov     ecx, r13d
36  and     ecx, ~7
37  test    ecx, ecx
38  jle     .cs_reduccionColaEscalar
39
40 .cs_bucle_vectorial:
41  cmp     eax, ecx
42  jge     .cs_reduccion_horizental
43  vmovups  ymm7, [r12 + rax*4]    ; ymm7 = arr[i..i+7]
44  vsubps   ymm6, ymm7, ymm2      ; ymm6 = x - mean (ymm7 queda intacto)
45  vfmadd231ps ymm3, ymm6, ymm6  ; sumsq += (x-mean)^2 en una operacion
46  vminps   ymm4, ymm4, ymm7      ; min por carril, SIN saltos
47  vmaxps   ymm5, ymm5, ymm7      ; max por carril, SIN saltos
48  add     eax, 8
49  jmp     .cs_bucle_vectorial

```

Listing 9: Prólogo, llamada a `sum_array` y bucle principal de `compute_stats` (versión vectorial)

La primera justificación necesaria en la presente rutina es la del ajuste `sub rsp, 8` antes del `call`, debido a que el ABI System V exige que `rsp` sea múltiplo de 16 bytes en el momento de ejecutar una instrucción `call`. De la misma manera, al momento de entrar a `compute_stats`, la dirección de retorno depositada por el `call` del `driver.c` deja al registro `rsp` desalineado en 8 bytes, ya que los seis `push` del prólogo añaden 48 bytes y los mismos al ser múltiplo de 16 no corrigen el desfase producido. De forma similar, la resta explícita de 8 bytes restaura la alineación exigida, y la instrucción `add rsp, 8` posterior la deshace para que los `pop` del epílogo recuperen los valores correctos del proyecto. A su vez, el omitir este ajuste no produce un fallo de forma inmediata, pero sí lo generaría en cuanto la función llamada ejecutara alguna instrucción SSE/AVX alineada sobre la pila, por lo esto si constituye una violación clara del ABI, la cual a su vez, debe corregirse por el diseño en si. Ahora bien, otra diferencia clave en comparación con la versión escalar es que en versión vectorial si se preservan los 6 registros y no 5 registros como en la versión escalar, porque aquí sí se emplea `rbp` como uno de los registros que contiene un puntero de salida.

La segunda justificación es la de poder reutilizar la rutina `sum_array` para calcular la media, en lugar de colocar la suma dentro del bucle principal como se efectuó en la versión escalar. La motivación de dicha decisión es la de mantenibilidad y de claridad, puesto que la reducción de la suma ya está implementada y verificada en dicha rutina, y la restricción algorítmica que presenta la ecuación (1) obliga a conocer la media completa, antes de poder efectuar alguna acumulación de

las diferencias al cuadrado. De la misma forma, es importante resaltar, que para la interpretación de los tiempos que se efectúa en la Sección 6, esta diferencia no desequilibra la comparación y el análisis efectuado, ya que contando la llamada que efectuó el archivo `driver.c` que efectúa sobre la rutina `sum_array` antes de la rutina `compute_stats`, se puede afirmar que ambas versiones realizan el mismo número de recorridos completos del arreglo (en el presente proyecto son 3 recorridos) antes de la rutina de normalización.

Por otra parte, la instrucción `vbroadcastss` permite operar un escalar contra un vector, puesto que las instrucciones empaquetadas operan carril a carril y el restar la media a los 8 elementos del registro `ymm7` requiere que la media esté presente en los 8 carriles del registro `ymm2`, ya que este comportamiento también lo efectúa la instrucción `vbroadcastss ymm2, xmm0`, replicando de esta forma el valor de 32 bits 8 veces. Asimismo, este mismo mecanismo se va a emplear para emplear el mínimo y el máximo con `arr[0]`, puesto que lee directamente desde memoria con el uso de la instrucción `vbroadcastss ymm4, dword [r12]`. De forma similar, la elección de `arr[0]` como semilla en lugar de 0, responde directamente a la misma razón expuesta en la Sección 3.3, la cual indica que un arreglo de datos compuesto exclusivamente por valores negativos reportaría un máximo de 0,0 si se empleara con 0.

Por el contrario, el bucle principal de la presente rutina refleja claramente la ventaja de la codificación VEX de 3 operandos anunciada en la Sección 4.1, puesto que la instrucción `vsubps ymm6, ymm7, ymm2` guarda la diferencia en el registro `ymm6` sin destruir los datos almacenados en el registro `ymm7`, de modo que las instrucciones `vminps` y `vmaxps` siguientes pueden consultar directamente los datos originales sin la necesidad de ninguna copia intermedia. Ahora bien, con la codificación SSE habrían sido necesarias y de forma obligatoria al menos dos instrucciones `movaps` adicionales por iteración solo para poder conservar el vector de entrada.

De la misma manera, es importante destacar que las instrucciones `vminps` y `vmaxps` resuelven el mínimo y el máximo, sin la necesidad de ningún salto condicional, a diferencia de las instrucciones `comiss + jae/jbe` las cuales fueron empleadas en la versión escalar. A su vez, esto elimina del bucle dos ramas dependientes de los datos, las cuales en un arreglo de datos de forma aleatoria son esencialmente impredecibles y las mismas pueden generar fallos de predicción de salto. Asimismo, esto genera una fuente de ganancia adicional para el presente proyecto, puesto que independientemente del ancho del registro el cual conviene tener presente al interpretar el *speedup* observado, esto implica que la aceleración medida no es atribuible únicamente al factor 8 del paralelismo de datos.

De forma similar, la instrucción `vfmadd231ps ymm3, ymm6, ymm6` calcula $ymm3 \leftarrow ymm6 \times ymm6 + ymm3$, la que acumula las 8 diferencias al cuadrado en una única instrucción que llega a sustituir las instrucciones `vmulps + vaddps`. Asimismo, el sufijo 231 indica el orden que se debe tener en la codificación, la cual asigna los operandos de la siguiente manera: el segundo y el tercero se multiplican y el primero actúa como un acumulador y el destino.

Una vez que el bucle vectorial termina de recorrerse por completo, los 3 acumuladores los cuales están presentes en el registro `ymm3` para la suma de cuadrados, en el registro `ymm4` para el dato mínimo y en el registro `ymm5` para dato el máximo, todavía contienen 8 valores parciales independientes, uno por cada carril, los cuales a su vez, se deben colapsarse a un único valor escalar antes de poder utilizarlos para combinarlos con el remanente escalar o escribirlos en memoria. Asimismo, el mecanismo general de esta reducción en específico ya se describió de forma conceptual o teórica en la Sección 4.6, a continuación se presenta el fragmento del código real que lo implementa dentro de `compute_stats`.

```

1 .cs_reduccion_horizontal:
2   ; sumsq: 8 carriles -> escalar (mismo patron que sum_array)
3   vextractf128 xmm8, ymm3, 1 ; xmm8 = mitad alta de ymm3 (carriles 4-7)
4   vaddps xmm3, xmm3, xmm8 ; xmm3 = 4 sumas parciales
5   vhaddps xmm3, xmm3, xmm3 ; reduce de 4 a 2

```

```

6  vhaddps xmm3, xmm3, xmm3      ; xmm3[0] = sumsq total (8 carriles)
7
8  ; min: 8 carriles -> escalar
9  vextractf128 xmm8, ymm4, 1    ; xmm8 = mitad alta (carriles 4-7)
10 vminps  xmm4, xmm4, xmm8      ; min entre carriles 0-3 y 4-7 -> 4 valores
11 vmovhlps xmm8, xmm4, xmm4     ; xmm8 = carriles altos de xmm4 movidos a la parte baja
12 vminps  xmm4, xmm4, xmm8      ; min entre pares -> 2 valores
13 vmovshdup xmm8, xmm4         ; xmm8 = carril impar duplicado
14 vminps  xmm4, xmm4, xmm8      ; xmm4[0] = minimo final (8 carriles)
15
16 ; max: 8 carriles -> escalar (misma tecnica que min)
17 vextractf128 xmm8, ymm5, 1
18 vmaxps  xmm5, xmm5, xmm8
19 vmovhlps xmm8, xmm5, xmm5
20 vmaxps  xmm5, xmm5, xmm8
21 vmovshdup xmm8, xmm5
22 vmaxps  xmm5, xmm5, xmm8      ; xmm5[0] = maximo final (8 carriles)

```

Listing 10: Reducción horizontal de `compute_stats`: de 8 carriles a un escalar (suma de cuadrados, mínimo y máximo)

Es importante destacar que en diferencia de la suma, para la cual la instrucción `vhaddps` realiza directamente la suma de pares adyacentes de datos, no existe una instrucción de reducción horizontal nativa para poder obtener el dato mínimo y el dato máximo. Debido a esta anterior razón, el registro `xmm8` se reutiliza como un registro temporal en las tres reducciones: primeramente en la instrucción `vextractf128` la cual lleva los 8 carriles a 4 carriles (combinando de esta manera la mitad alta con la baja), luego la instrucción `vmovhlps` la cual reordena los 2 carriles altos de esos 4 hacia la posición baja del registro `xmm8` para poder compararlos contra los dos carriles bajos originales (reduciendo de 4 a 2), y finalmente la instrucción `vmovshdup` la cual tiene como objetivo duplicar el carril impar restante para la última comparación (de 2 a 1). De la misma manera, es importante recordar que al finalizar esta etapa, los diversos valores reducidos quedan en el elemento menos significativo de los mismos registros los cuales antes actuaban como acumuladores de 256 bits (`xmm3`, `xmm4`, `xmm5` son, respectivamente, la mitad baja de los registros `ymm3`, `ymm4` y `ymm5`), por lo cual no se necesita ninguna instrucción de tipo `mov` adicional para de esta manera poder continuar operando con ellos como escalares en la etapa siguiente.

```

1  .cs_reduccionColaEscalar:
2      cmp     eax, r13d
3      jge     .cs_almacenar_resultados
4      vmovss  xmm9, [r12 + rax*4]      ; xmm9 = arr[i]
5      vsubss  xmm10, xmm9, xmm0        ; xmm10 = x - mean (mean sigue en xmm0)
6      vmulss  xmm10, xmm10, xmm10      ; xmm10 = (x - mean)^2
7      vaddss  xmm3, xmm3, xmm10        ; sumsq += (x - mean)^2
8      vminss  xmm4, xmm4, xmm9        ; min = min(min, x)
9      vmaxss  xmm5, xmm5, xmm9        ; max = max(max, x)
10     inc     eax
11     jmp     .cs_reduccionColaEscalar

```

Listing 11: Reducción de cola escalar de `compute_stats`: elementos remanentes ($n \bmod 8$)

Finalmente, el bucle de cola retoma el índice que se almacena en el registro `eax` exactamente donde lo dejó el bucle vectorial (en el registro `ecx` y por el cual es el múltiplo de 8 más cercano por abajo a n) y a su vez recorre uno a uno por todos los elementos restantes hasta `r13d` (n original), la cual a su vez sigue la misma estrategia de dos fases que se presentó y describió en la Sección 4.7. De forma similar, cada instrucción empaquetada del bucle principal de esta rutina se sustituye en esta parte por su contraparte escalar que tiene el sufijo `ss` (`vsubss`, `vmulss`, `vaddss`, `vminss`, `vmaxss`), y los tres acumuladores continúan siendo los registros `xmm3`, `xmm4` y `xmm5`, de modo que cuando la ejecución del código llegue a `.cs_almacenar_resultados` ya debe de contener el resultado

combinado de los diversos elementos vectorizados y del remanente escalar, sin necesidad alguna de fusionar dos resultados por forma separada.

Caso borde $n = 0$ y cierre de la rutina

```

1 .cs_almacenar_resultados:
2     cvtsi2ss xmm1, r13d           ; n como float
3     divss    xmm3, xmm1           ; var = sumsq / n
4     movss    [rbp], xmm3         ; *var
5     movss    [r14], xmm4         ; *min
6     movss    [r15], xmm5         ; *max
7     jmp      .cs_cierre
8
9 .cs_vaciar_registros:             ; caso borde n <= 0
10    xorps    xmm0, xmm0
11    movss    [rbx], xmm0          ; los cuatro resultados quedan en 0.0,
12    movss    [rbp], xmm0          ; tal como exige el contrato de stats.h
13    movss    [r14], xmm0
14    movss    [r15], xmm0
15
16 .cs_cierre:
17    vzeroupper
18    pop      r15
19    pop      r14
20    pop      r13
21    pop      r12
22    pop      rbx
23    pop      rbp
24    ret

```

Listing 12: Almacenamiento de resultados, caso borde y epílogo de `compute_stats` (versión vectorial)

En primer lugar, el caso $n \leq 0$ conserva la misma estructura de la versión escalar, puesto que la comprobación se realiza con `test` antes de cualquier tipo de acceso a memoria, lo cual es sumamente imprescindible porque la siembra del mínimo y del máximo lee lo que almacena `arr[0]` y a su vez la media divide entre n . Asimismo, la rama de vaciado cae por caída natural o consecuencia sobre `.cs_cierre`, lo que a su vez garantiza un punto de salida único en el que se pueden restaurar los 6 registros (denominados callee-saved) en un orden inverso al de los `push` y finalmente se ejecuta `vzeroupper`.

4.5. `normalize_array` vectorizado

Esta rutina en particular se puede afirmar que es la más favorable para el proceso de vectorización en comparación al resto del proyecto, porque no requiere de ninguna tipo de reducción, puesto que se trata de una operación puramente elemento a elemento en la que los 8 carriles son completamente independientes entre sí y a su vez, el resultado se escribe directamente en el arreglo de salida.

```

1 normalize_array:
2     xorps    xmm4, xmm4
3     comiss   xmm1, xmm4           ; comprobacion UNICA de stddev == 0
4     je       .na_camino_copia     ; antes de entrar a ningun bucle
5
6     vbroadcastss ymm2, xmm0       ; mean en 8 carriles
7     vbroadcastss ymm3, xmm1       ; stddev en 8 carriles
8
9     xor      eax, eax
10    mov      ecx, edx
11    and      ecx, ~7
12    test     ecx, ecx
13    jle      .na_bucle_escalar_cierre
14

```

```

15 .na_bucle_vectorial:
16     cmp     eax, ecx
17     jge     .na_bucle_escalador_cierre
18     vmovaps ymm5, [rdi + rax*4]      ; carga ALINEADA de 8 floats
19     vsubps  ymm5, ymm5, ymm2        ; x - mean
20     vdivps  ymm5, ymm5, ymm3        ; (x - mean) / stddev
21     vmovaps [rsi + rax*4], ymm5     ; escritura alineada de 8 floats
22     add     eax, 8
23     jmp     .na_bucle_vectorial
24
25 .na_bucle_escalador_cierre:          ; remanente n % 8
26     cmp     eax, edx
27     jge     .na_cierre
28     vmovss  xmm6, [rdi + rax*4]
29     subss   xmm6, xmm0
30     divss   xmm6, xmm1
31     vmovss  [rsi + rax*4], xmm6
32     inc     eax
33     jmp     .na_bucle_escalador_cierre
34
35 .na_cierre:
36     vzeroupper
37     ret

```

Listing 13: Fragmento clave de `normalize_array` (versión vectorial AVX2)

En esta rutina al igual que en su versión escalar, la verificación de $\sigma = 0$ se realiza una sola vez antes de poder entrar a cualquier tipo de bucle, ya que `stddev` es un argumento el cual no varía o cambia durante toda la ejecución. La rutina presenta dos caminos mutuamente excluyentes y el camino de copia directa (`.na_camino_copia`) está a su vez vectorizado, lo cual es una copia de bloques de 8 floats de `in` hacia `out` sin necesidad de ningún cálculo aritmético adicional. Asimismo una diferencia clara es que no se efectúa una copia de forma puramente escalar, la cual se va realizando de elemento por elemento como se presentó en la versión escalar del presente proyecto.

```

1 .na_camino_copia:
2     xor     eax, eax                ; eax = i = 0
3     mov     ecx, edx                ; ecx = n
4     and     ecx, ~7                ; ecx = n & ~7 (redondeo hacia abajo al múltiplo de 8)
5     test    ecx, ecx
6     jle     .na_copiarCola         ; si n < 8, todo el arreglo se copia escaladamente
7
8 .na_copiar_vectorial:
9     cmp     eax, ecx
10    jge     .na_copiarCola
11    vmovaps ymm5, [rdi + rax*4]     ; ymm5 = in[i..i+7] (carga alineada)
12    vmovaps [rsi + rax*4], ymm5     ; out[i..i+7] = in[i..i+7] (escritura alineada)
13    add     eax, 8
14    jmp     .na_copiar_vectorial
15
16 .na_copiarCola:                    ; remanente n % 8
17     cmp     eax, edx
18     jge     .na_cierre
19     vmovss  xmm6, [rdi + rax*4]     ; xmm6 = in[i]
20     vmovss  [rsi + rax*4], xmm6     ; out[i] = in[i]
21     inc     eax
22     jmp     .na_copiarCola

```

Listing 14: Camino de copia directa de `normalize_array` (caso $\sigma = 0$)

Es importante resaltar, que este camino recalcula `ecx = edx & ~7` de forma independiente al que ya se calculó en el camino normal, puesto que el mismo que se observa en `.na_camino_copia` se alcanza mediante el uso de un salto `je` antes de que se ejecute cualquier otra instrucción del camino normal, incluyendo el cálculo de dicho límite, esto es debido a que los dos caminos son completamente autónomos entre sí y no se comparten estado alguno más allá de los argumentos de la entrada. De

la misma manera, la carga como el proceso de almacenamiento del bucle vectorial usan la variante alineada de la instrucción `vmovaps`, bajo el mismo razonamiento que se describió en la Sección 4.8 la cual hace énfasis a la garantía de 32 bytes que se otorga en `aligned_alloc` que esta alojada en el `driver.c`. Finalmente, `.na_copiarCola` como `.na_bucle_escalar_cierre` del camino normal ambas terminan cayendo en la misma etiqueta `.na_cierre`, lo que a su vez garantiza que exista un único punto de salida de la función actual y que la instrucción `vzeroupper` se ejecute sin importar cuál de los dos caminos se haya tomado.

Por otra parte, esta decisión de diseño resulta fundamental para la interpretación de los diversos resultados de la Sección 6, puesto que el caso de prueba de valores constantes ($x_i = 5$, $N = 1000$, el cual se puede observar en la Tabla 5) desarrolla exactamente esta sección del código, debido a que al estar vectorizada de la misma manera que el camino normal, el tiempo de ejecución de dicho caso no se va a ver penalizado por haber caído en el caso borde $\sigma = 0$, manteniendo de esta forma la comparación de las diversas mediciones de rendimiento entre ambas rutas de ejecución.

4.6. Estrategia de reducción horizontal

Las tres funciones necesitan colapsar un acumulador de 8 carriles (`ymm`) a un único valor escalar antes de poder combinarlo con el remanente o escribirlo en memoria. Para la suma (`sum_array` y el acumulador `sumsq` de `compute_stats`) se usa la secuencia `vextractf128 + vaddps + doble vhaddps`: primero se extrae la mitad alta del `ymm` (carriles 4–7) a un `xmm` y se suma contra la mitad baja, reduciendo de 8 a 4 sumas parciales en 128 bits; luego dos `vhaddps` consecutivos reducen de 4 a 2 y de 2 a 1. Esta técnica se prefirió por ser la de menor número de instrucciones para una operación asociativa y conmutativa como la suma, sin necesitar shuffles explícitos.

Para mínimo y máximo no existe una instrucción de reducción horizontal nativa, por lo que la reducción se construyó manualmente: `vextractf128 + vminps/vmaxps` reduce de 8 a 4 carriles; `vmovhlps` mueve los dos carriles altos de esos 4 a la parte baja de un registro temporal para comparar contra los dos carriles bajos, reduciendo a 2; y `vmovshdup` duplica el carril impar inferior para compararlo contra el par, reduciendo el último a 1. A diferencia de `vhaddps` (que suma pares adyacentes), `vminps/vmaxps` solo comparan carril a carril entre dos registros, por lo que la reducción de mínimo/máximo requiere reordenar explícitamente los carriles con `vmovhlps/vmovshdup` en lugar de una instrucción horizontal directa.

4.7. Manejo del remanente (tail loop)

Como AVX2 procesa 8 floats por iteración, si n no es múltiplo de 8 quedan $n \bmod 8$ elementos que no completan un vector. Las tres funciones resuelven esto con el mismo patrón de dos fases: primero `ecx = n & ~7` calcula cuántos elementos entran en bloques completos de 8 (redondeo hacia abajo), y el bucle vectorial procesa únicamente ese rango (`i` de 0 a `ecx`, de 8 en 8). Al salir de ese bucle, un segundo bucle puramente escalar procesa los elementos restantes uno a uno, con `i` recorriendo desde `ecx` hasta el n original, usando las versiones escalares de las mismas instrucciones (`vaddss/addss`, `vsubss/subss`, `vminss`, `vmaxss`, etc.). Este bucle de cola también cubre, como caso extremo, los arreglos con $n < 8$: en ese caso `ecx` queda en 0, el bucle vectorial no ejecuta ninguna iteración, y todo el arreglo se procesa escalaramente.

4.8. Alineación de memoria

`driver.c` reserva los arreglos de entrada y salida alineados a 32 bytes con `aligned_alloc`, cumpliendo el requisito de AVX2 para operar sobre bloques de 256 bits sin generar una excepción de alineación. En `normalize_array` esta garantía se aprovecha directamente: tanto la carga (`vmovaps`

`ymm5, [rdi + rax*4]`) como el almacenamiento (`vmovaps [rsi + rax*4], ymm5`) en el bucle vectorial usan la variante alineada, ya que el desplazamiento `rax*4` avanza en múltiplos de 32 bytes (`rax` incrementa de 8 en 8) sobre una dirección base que ya es múltiplo de 32.

En `sum_array` y en el bucle vectorial de `compute_stats`, en cambio, las cargas se hacen con `vmovups` (no alineada) a pesar de que la misma garantía de alineación aplica sobre `arr`. Esta elección es más conservadora que necesaria — `vmovaps` habría sido válida en ambos casos, dado el mismo razonamiento de offset múltiplo de 32 — y es un punto que conviene poder justificar en la entrevista: la decisión no afecta la correctud, pero sí representa una oportunidad de rendimiento no aprovechada, ya que `vmovups` puede tener una latencia ligeramente mayor que `vmovaps` sobre datos ya alineados en algunas microarquitecturas.

5. Casos de prueba y verificación de correctud

Caso de prueba	Entrada	Referencia	Escalar / Vectorial	Resultado
Arreglo vacío	$N = 0$	$\mu = 0, \sigma^2 = 0$	Coinciden	Aprobado
Un elemento	$N = 1$	$\mu = 27,8854, \sigma^2 = 0$	Coinciden	Aprobado
Tamaño no múltiplo de 8	Aleatorio, $N = 15$	$\mu = -25,2542, \sigma^2 = 3169,04$	Coinciden	Aprobado
Valores constantes	$x_i = 5, N = 1000$	$\mu = 5, \sigma^2 = 0$	Coinciden	Aprobado
Valores negativos	$x_i \in [-500, -0,001], N = 64$	$\mu = -266,493, \sigma^2 = 20437,1$	Coinciden	Aprobado
Valores extremos	$x_i \in [-10^6, 10^6], N = 100$	$\mu = 0, \sigma^2 \approx 3 \times 10^{11}$	Coinciden	Aprobado

Cuadro 5: Resultados de los casos de prueba para las implementaciones escalar y vectorial.

La comparación con la implementación de referencia se automatizó mediante el script de python `verify_reference.py`. Este ejecuta las versiones escalar y vectorial para cada caso de prueba y contrasta los estadísticos obtenidos con los calculados mediante NumPy. Debido a las diferencias inherentes a la aritmética de punto flotante, se estableció una tolerancia relativa de 10^{-4} . Los resultados se consideran correctos cuando la media, varianza, mínimo y máximo satisfacen este criterio de tolerancia respecto a los valores de referencia.

6. Resultados de rendimiento

6.1. Tiempos de ejecución

N	Tiempo escalar (ms)	Desv. std.	Tiempo vectorial (ms)	Desv. std.
10^3	0,009997	0,000306	0,001491	0,0000284
10^5	0,968433	0,042371	0,117189	0,014631
10^6	10,029485	0,610540	1,672887	0,425531
5×10^7	247,625756	7,348476	67,059873	2,953542

Cuadro 6: Tiempo promedio de ejecución del kernel (30 invocaciones independientes por tamaño) y desviación estándar.

6.2. Speedup

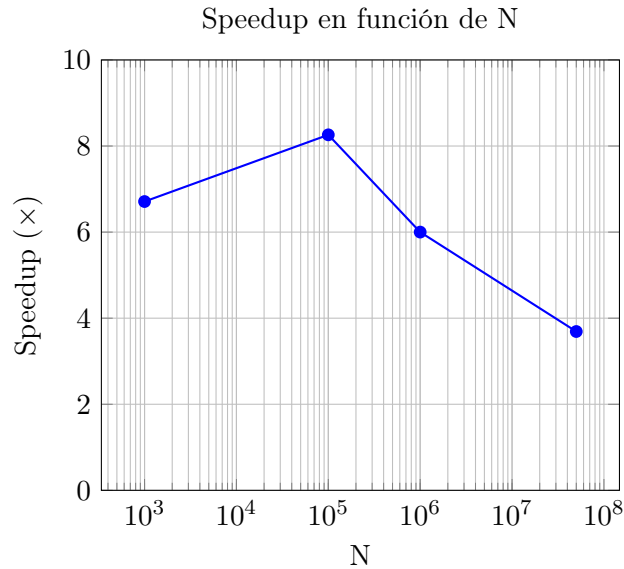


Figura 1: Gráfica de Speedup ($t_{escalar}/t_{vectorial}$) según el tamaño de la muestra N.

El speedup no es monótono (Figura 1): crece de $6,7 \times$ ($N = 10^3$) a $8,3 \times$ ($N = 10^5$) y luego cae a $6,0 \times$ ($N = 10^6$) y $3,7 \times$ ($N = 5 \times 10^7$). La interpretación se apoya en la relación entre el conjunto de trabajo de cada tamaño (arreglos de entrada y salida, $8N$ bytes) y la jerarquía de memoria del Ryzen 5 7520U (Zen 2): 32 KB de L1 de datos y 512 KB de L2 por núcleo, y 4 MB de L3 compartida (Tabla 7).

N	Conjunto de trabajo (in + out)	Nivel de memoria dominante
10^3	≈ 8 KB	L1
10^5	≈ 800 KB	L2/L3 (la entrada sola, 400 KB, cabe en L2)
10^6	≈ 8 MB	L3 saturada y RAM (la entrada sola ocupa toda la L3)
5×10^7	≈ 400 MB	RAM

Cuadro 7: Conjunto de trabajo por tamaño frente a la jerarquía de memoria.

Crecimiento de 10^3 a 10^5 : Con $N = 10^3$ todo reside en L1, por lo que el kernel está limitado por cómputo y el ancho de 8 carriles es el techo. Sin embargo, el tiempo total vectorial es de apenas $\approx 1,5 \mu\text{s}$, de modo que los costos fijos (prólogo y epílogo, reducciones horizontales en `sum_array` y `compute_stats`, llamadas entre rutinas y la propia medición con `clock_gettime`) pesan una fracción mayor y reducen el speedup a $6,7\times$. A $N = 10^5$ esos costos se amortizan y el speedup alcanza $8,3\times$, valor compatible con el techo de $8\times$ dada la dispersión de las mediciones (la desviación estándar vectorial es 12.5 % de su promedio).

Caída para $N \geq 10^6$: Al superar la capacidad de la L3 (4 MB), los accesos pasan a memoria principal y el kernel vectorial, que consume datos mucho más rápido, empieza a esperar a la memoria; el escalar, más lento por elemento, todavía no está limitado por ella. Con $N = 5 \times 10^7$ el conjunto de trabajo es de 400 MB y cada invocación lee la entrada cuatro veces (`sum_array`, la llamada interna de `compute_stats`, su bucle fusionado y `normalize_array`) y escribe la salida una vez, lo que implica al menos 1 GB de tráfico. Con los tiempos medidos, el ancho de banda efectivo es de $\approx 14 \text{ GB/s}$ en la versión vectorial y $\approx 4 \text{ GB/s}$ en la escalar. La versión escalar sigue limitada por la ejecución de instrucciones, mientras que la vectorial se aproxima al límite del sistema de memoria, por lo que el speedup tiende a la razón de anchos de banda alcanzados ($3,7\times$) y no a la razón de carriles ($8\times$). La alta dispersión del vectorial a $N = 10^6$ (25 %) es compatible con operar en el borde de capacidad de la L3, donde la tasa de aciertos varía entre corridas; por ello ese punto tiene mayor incertidumbre. Esta interpretación se contrasta con `perf stat` (IPC y fallos de caché) en la subsección siguiente.

6.3. Análisis con `perf stat`

Se ejecutó `perf stat -e cycles,instructions,cache-misses` sobre ambos binarios con 30 repeticiones internas del kernel por proceso. El Listado 15 muestra la salida para $N = 5 \times 10^7$ y la Tabla 8 resume los cuatro tamaños.

```

1 Performance counter stats for './bin/norm_scalar data/perf_input_50000000.dat data/
2   perf_output_scalar_50000000.dat 30':
3
4     33 285 750 584      cycles
5     48 908 671 284      instructions
6     23 120 957          cache-misses
7
8     7,892173517 seconds time elapsed
9
10    7,406120000 seconds user
11    0,364005000 seconds sys
12
13 Performance counter stats for './bin/norm_vector data/perf_input_50000000.dat data/
14   perf_output_vector_50000000.dat 30':
15
16    10 832 755 520      cycles
17     6 313 205 345      instructions
18    28 707 788          cache-misses
19
20    2,658403628 seconds time elapsed
21
22    2,171745000 seconds user
23    0,367121000 seconds sys

```

Listing 15: Salida de `perf stat -e cycles,instructions,cache-misses` para ambas versiones ($N = 5 \times 10^7$)

N	Versión	Ciclos	Instrucciones	IPC	Cache-misses	% kernel
10^3	Escalar	4 572 164	2 278 520	0.50	50 173	5.2
	Vectorial	3 805 512	1 445 303	0.38	54 624	0.7
10^5	Escalar	67 922 099	99 262 137	1.46	113 270	81.5
	Vectorial	15 459 616	13 985 056	0.90	113 504	31.1
10^6	Escalar	660 447 399	978 824 149	1.48	526 017	93.6
	Vectorial	183 968 102	126 688 550	0.69	604 406	78.3
5×10^7	Escalar	33 285 750 584	48 908 671 284	1.47	23 120 957	93.7
	Vectorial	10 832 755 520	6 313 205 345	0.58	28 707 788	81.4

Cuadro 8: Contadores de `perf stat` del proceso completo (30 repeticiones del kernel). La columna % kernel es $30 t_{\text{kernel}}/t_{\text{transcurrido}}$; cuando es baja, el IPC refleja sobre todo el arranque y la E/S.

Alcance de los contadores. `perf stat` mide el proceso completo (arranque, lectura del archivo, fallos de página y escritura de la salida), no solo el kernel. Para $N = 10^3$ (menos de 6 %) y para el vectorial con $N = 10^5$ (31 %) el IPC refleja sobre todo el costo de arranque, por lo que el análisis se apoya en $N \geq 10^6$ y en el escalar con $N = 10^5$, donde el kernel supera el 78 % del tiempo transcurrido.

Comparación de IPC. El IPC escalar es prácticamente constante (1.46 a 1.48) de 10^5 a 5×10^7 , y su tiempo por elemento casi no cambia (4.7 a 4.9 ns) al pasar de datos en L2/L3 a RAM. Ejecuta ≈ 33 instrucciones por elemento y repetición, y su rendimiento no depende de la jerarquía de memoria, lo que indica un cuello de botella de cómputo. El IPC vectorial es menor (0.69 con $N = 10^6$ y 0.58 con $N = 5 \times 10^7$), decrece al crecer el conjunto de trabajo y coincide con un aumento del tiempo por elemento de 1.17 a 1.44 ns. El vectorial ejecuta $\approx 4,2$ instrucciones por elemento y repetición.

Descomposición de la ganancia. La razón de ciclos entre versiones es el producto de la reducción de instrucciones por la razón de IPC:

$$\frac{\text{ciclos}_{\text{esc}}}{\text{ciclos}_{\text{vec}}} = \frac{\text{instr}_{\text{esc}}}{\text{instr}_{\text{vec}}} \cdot \frac{\text{IPC}_{\text{vec}}}{\text{IPC}_{\text{esc}}}$$

Con $N = 5 \times 10^7$: $3,07 = 7,75 \times 0,40$. Con $N = 10^6$: $3,59 = 7,73 \times 0,46$. La vectorización logra la reducción de instrucciones esperada (cercana a 8; la diferencia es compatible con el costo común de arranque y E/S, cuyo tiempo de sistema es casi idéntico en ambas versiones: 0.364 s y 0.367 s). Sin embargo, el IPC vectorial es de 2.2 a 2.5 veces menor que el escalar, y ese factor hace que la ganancia real sea de 3 a 3.6 veces y no de 8.

Cómputo frente a ancho de banda. Cada repetición mueve $20N$ bytes (cuatro lecturas y una escritura de N floats). Con los tiempos medidos en estas corridas, el rendimiento efectivo del escalar es $\approx 4,1$ GB/s en todos los tamaños, mientras que el vectorial pasa de ≈ 35 GB/s ($N = 10^5$) a ≈ 17 GB/s ($N = 10^6$) y ≈ 14 GB/s ($N = 5 \times 10^7$). El escalar está limitado por cómputo en todos los tamaños. El vectorial lo está mientras el conjunto de trabajo reside en caché y pasa a estar limitado por el ancho de banda de memoria cuando lo supera (desde $N = 10^6$), lo que es coherente con la interpretación de la subsección anterior.

Los fallos de caché apuntan en la misma dirección. Con $N = 5 \times 10^7$ son 23.1 millones (escalar) y 28.7 millones (vectorial), es decir 0.47 y 4.5 por cada mil instrucciones: el vectorial genera ≈ 10 veces más fallos por instrucción porque cada instrucción consume 8 elementos. En ambas versiones los fallos por repetición (0.8 a 1.0 millones) son una fracción pequeña de las $\approx 15,6$ millones de líneas de 64 bytes que recorre cada repetición, lo que es compatible con la acción del prefetcher de

hardware sobre accesos secuenciales. Por ello **cache-misses** se interpreta como indicador relativo y no como medida del tráfico total.

Por qué el speedup no alcanza $8\times$. (i) *Ancho de banda*: es el factor dominante para $N \geq 10^6$, donde el vectorial deja de estar limitado por cómputo. (ii) *Costos fijos y Ley de Amdahl*: la parte no vectorizable del kernel (prólogo y epílogo, replicación de escalares con **vbroadcastss**, reducciones horizontales y llamadas entre rutinas) es constante, por lo que su peso relativo crece cuando N es pequeño y contribuye a que con $N = 10^3$ el speedup sea menor que 8. Las reducciones horizontales son cadenas de instrucciones dependientes (**vextractf128**, **vaddps**, dos **vhaddps**) cuya latencia no se amortiza con pocas iteraciones. A escala de programa, la parte no acelerada (arranque, lectura y escritura de archivos, fallos de página) es $\approx 0,49$ s en ambas versiones con $N = 5 \times 10^7$. Sobre el tiempo transcurrido (7.89 s escalar y 2.66 s vectorial) el speedup del programa completo es $2,97\times$, inferior al $3,42\times$ del kernel, y aunque el kernel alcanzara $8\times$ el programa completo llegaría a $\approx 5,6\times$. (iii) *Latencia de instrucciones*: la vectorización divide por ≈ 8 el número de instrucciones, pero no la latencia de cada una. Los acumuladores de las reducciones forman cadenas de dependencia y **vdivps** en **normalize_array** tiene alta latencia y bajo throughput, lo que puede limitar el rendimiento aun con los datos en caché.

7. Evidencia de la sesión de depuración con GDB

En primer lugar, a diferencia de la Sección 6, en la cual la medición de los diversos tiempos se automatiza mediante el uso de un script externo denominado **test_performance.sh** y que el mismo se ejecuta mediante el uso de 30 invocaciones independientes por tamaño y versión, pero cuyo instrumento real de medición sigue siendo **clock_gettime** que esta alojada dentro del **driver.c**, y en la cual el análisis de IPC y fallos de caché se obtiene mediante el uso de **perf stat**, en comparación con la presente sección GDB no cumple ningún papel de temporización. De la misma manera, su único propósito u objetivo es el poder verificar de forma funcional, a nivel de cada uno de los registro y de la memoria, de que la ejecución de la versión vectorizada correspondería exactamente al comportamiento que se diseñó (en otras palabras al comportamiento que se espera) y se justificó en la Sección 4. Asimismo, para lograr esto se compiló el proyecto con las distintas banderas de depuración que ya incluye el **Makefile** (**nasm -f elf64 -g -F dwarf** y **gcc -g**), lo cual a su vez permitió que la herramienta de GDB se pueda asociar con cada una de las distintas direcciones de memoria del binario **bin/norm_vector** y con su línea correspondiente dentro de **asm/vector/stats_vector.asm**.

Por otra, el análisis se obtuvo mediante **tools/gen_input.py 16 data/input_gdb16.dat random 42**, y teniendo un caso de prueba específico con $N = 16$. Asimismo, esta elección no fue arbitraria, puesto que 16 es múltiplo exacto de 8 (el ancho del registro YMM en elementos de tipo **float**), lo cual a su vez garantiza que el bucle vectorial de **normalize_array** se pueda ejecutar exactamente dos veces y que el bucle escalar de cierre (Sección 4.7) no llegue a ejecutarse en ninguna iteración, de manera que toda la totalidad del arreglo de salida quede determinada únicamente por la ruta vectorial que se quiere verificar en esta sección. A su vez, el análisis se inició con la ejecución de la instrucción en la consola **gdb -args bin/norm_vector data/input_gdb16.dat data/output_gdb16.dat 1**.

7.1. Inspección de registro YMM tras **vaddps**

El primer punto de interrupción que se colocó para el presente análisis de GDB, fue justo después de la instrucción **vaddps ymm0, ymm0, ymm1** del Listado 8 dentro de la rutina **sum_array**, sobre la línea que contiene la instrucción **add eax, 8** inmediatamente posterior (es importante resaltar que en el binario compilado y en esta línea puntual cae justo antes del salto que se ejecutara con el uso de la instrucción **jmp .sum_vec_loop** la cual a su vez cierra la iteración). De la misma manera, la

rutina `sum_array` se invoca dos veces durante una corrida completa del `driver.c` (una primera vez de forma directa para poder calcular `sum`, y una segunda vez dentro de la rutina `compute_stats` para esta manera poder obtener la media, según se describió en la Sección 4.4), y dado que se escogió el caso específico de $n = 16$, el cual produce exactamente dos iteraciones del bucle vectorial por cada llamada, a su vez el breakpoint se dejó activo únicamente durante la primera invocación, capturando de esta forma sus dos aciertos, y se eliminó antes de que la llamada interna de `compute_stats` pudiera generar aciertos adicionales sobre el mismo punto.

```

1 (gdb) break stats_vector.asm:44
2 Breakpoint 1 at 0x195f: file asm/vector/stats_vector.asm, line 44.
3 (gdb) run
4 Starting program: /home/steven-ubuntu/Proyecto-Arquitectura/proyecto_vectorial/bin/
   norm_vector data/input_gdb16.dat data/output_gdb16.dat 1
5
6 This GDB supports auto-downloading debuginfo from the following URLs:
7   <https://debuginfod.ubuntu.com>
8 Enable debuginfod for this session? (y or [n]) y
9 Debuginfod has been enabled.
10 To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.
11 Downloading 53.04 K separate debug info for system-supplied DS0 at 0x7ffff7fbe000
12 [Thread debugging using libthread_db enabled]
13 Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".
14
15 Breakpoint 1, sum_array.sum_vec_loop () at asm/vector/stats_vector.asm:44
16 44      jmp      .sum_vec_loop
17 (gdb) print $ymm0.v8_float
18 $1 = {27.8853588, -94.9978485, -44.9941368, -55.3578529,
19      47.2942429, 35.3398972, 78.4359131, -82.612236}
20 (gdb) print $eax
21 $2 = 8
22 (gdb) continue
23 Continuing.
24
25 Breakpoint 1, sum_array.sum_vec_loop () at asm/vector/stats_vector.asm:44
26 44      jmp      .sum_vec_loop
27 (gdb) print $ymm0.v8_float
28 $3 = {12.2697229, -189.038406, -101.266541, -54.2867966,
29      -47.3985596, -24.8925743, 108.412796, -73.6239395}
30 (gdb) print $eax
31 $4 = 16
32 (gdb) delete breakpoints
33 Delete all breakpoints, watchpoints, tracepoints, and catchpoints? (y or n) y

```

Listing 16: Transcripción de GDB: contenido de `ymm0` tras `vaddps` ($N=16$)

```

steven-ubuntu@Ubuntu: ~/Proyecto-Arquitectura/proyecto_vectorial — gdb --args bin/...
~/Proyecto-Arquitectura/proyecto_vectorial

steven-ubuntu@Ubuntu:~/Proyecto-Arquitectura/proyecto_vectorial$ make
steven-ubuntu@Ubuntu:~/Proyecto-Arquitectura/proyecto_vectorial$ python3 tools/gen_input.py 16 data/input_gdb16.dat random 42
Generado 'data/input_gdb16.dat' con N=16, modo=random
steven-ubuntu@Ubuntu:~/Proyecto-Arquitectura/proyecto_vectorial$ python3 -c "
import struct
with open('data/input_gdb16.dat','rb') as f:
    n = struct.unpack('<i', f.read(4))[0]
    vals = struct.unpack('<{n}f', f.read(4*n))
print(n, vals)

16 (27.885358810424805, -94.99784851074219, -44.994136810302734, -55.357852935791016, 47.29424285888672, 35.33989715576172, 78.4359130859375, -82.61223602294922, -15.615635871887207, -94.04855786132812, -56.272403717041016, 1.071857677268982, -94.69280242919922, -60.23247146606445, 29.976886749267578, 8.988296508789062)
steven-ubuntu@Ubuntu:~/Proyecto-Arquitectura/proyecto_vectorial$ gdb --args bin/norm_vector data/input_gdb16.dat data/output_gdb16.dat 1
GNU gdb (Ubuntu 17.1-2ubuntu1) 17.1
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from bin/norm_vector...
(gdb) break stats_vector.asm:44
Breakpoint 1 at 0x195f: file asm/vector/stats_vector.asm, line 44.
(gdb) run
Starting program: /home/steven-ubuntu/Proyecto-Arquitectura/proyecto_vectorial/bin/norm_vector data/input_gdb16.dat data/output_gdb16.dat 1

This GDB supports auto-downloading debuginfo from the following URLs:
<https://debuginfod.ubuntu.com>
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.
Downloading 53.04 K separate debug info for system-supplied DSO at 0x7ffff7be000
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, sum_array.sum_vec_loop () at asm/vector/stats_vector.asm:44
44      jmp     sum_vec_loop
(gdb) print $ymm0.v8_float
$1 = {27.8853588, -94.9978485, -44.9941368, -55.3578529, 47.2942429, 35.3398972, 78.4359131, -82.612236}
(gdb) print $eax
$2 = 8
(gdb) continue
Continuing.

Breakpoint 1, sum_array.sum_vec_loop () at asm/vector/stats_vector.asm:44
44      jmp     sum_vec_loop
(gdb) print $ymm0.v8_float
$3 = {12.2697229, -189.038406, -101.266541, -54.2867966, -47.3985596, -24.8925743, 108.412796, -73.6239395}
(gdb) print $eax
$4 = 16
(gdb) delete breakpoints
Delete all breakpoints, watchpoints, tracepoints, and catchpoints? (y or n) y

```

Figura 2: Captura de la sesión de GDB donde se evidencia el contenido de `ymm0` tras la instrucción `vaddps` en `sum_array` ($N=16$).

Por otra parte, el primer acierto con el registro `$eax = 8`, muestra que el registro `ymm0` contiene exactamente los primeros 8 elementos del arreglo de entrada sin modificación alguna, puesto que el acumulador se inicializó en cero mediante el uso de la instrucción `vxorps ymm0, ymm0, ymm0` (Listado 8). Asimismo, el segundo acierto, con el registro `$eax = 16`, llega a confirmar la acumulación elemento a elemento que se describe en la Sección 4.6 del presente proyecto, puesto que cada carril de `$3` corresponde a la suma del carril homólogo de `$1` más el elemento `arr[i+8]` correspondiente (en otras palabras lo anterior se puede evidenciar de mejor manera con el siguiente ejemplo: el carril 0 pasa de 27,8853588 a 12,2697229, lo que implica que `arr[8] = -15,6156`, valor que coincide con el noveno elemento del arreglo generado y mostrado en la captura). De forma similar, esto al mismo tiempo verifica que en este punto el registro `ymm0` todavía contiene 8 sumas parciales de forma independiente y no es un escalar reducido, tal como lo exige el diseño de la reducción horizontal que se ejecutara únicamente después de la etiqueta `.sum_vec_loop`.

7.2. Inspección de memoria del arreglo de salida

El segundo punto de interrupción que se utilizó para el análisis del GDB se ubicó en el `driver.c:171`, inmediatamente después de la llamada a `normalize_array(in, out, n, mean, stddev_r)`, en otras palabras una vez que el bucle vectorial de la Sección 4.5 ya terminó de escribir el arreglo de salida.

```

1 (gdb) break driver.c:171
2 Breakpoint 1 at 0x12fa: file src/driver.c, line 171.
3 (gdb) run data/input_gdb16.dat data/output_gdb16.dat 1
4 Starting program: /home/steven-ubuntu/Proyecto-Arquitectura/proyecto_vectorial/bin/
   norm_vector data/input_gdb16.dat data/output_gdb16.dat 1
5
6 This GDB supports auto-downloading debuginfo from the following URLs:
7   <https://debuginfod.ubuntu.com>
8 Enable debuginfod for this session? (y or [n]) y
9 Debuginfod has been enabled.
10 To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.
11 [Thread debugging using libthread_db enabled]
12 Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".
13
14 Breakpoint 1, main (argc=<optimized out>, argv=<optimized out>) at src/driver.c:171
15 171      clock_gettime(CLOCK_MONOTONIC, &t1); // F: final del programa (end)
16 (gdb) print out
17 $1 = (float *) 0x555555559520
18 (gdb) x/8fw out
19 0x555555559520:  0.925018966  -1.30381823  -0.396858305  -0.58483386
20 0x555555559530:  1.27705455   1.06022835   1.84189725  -1.07916975
21 (gdb) continue
22 Continuing.
23 N          = 16
24 Suma       = -369.824280
25 Media     = -23.114017
26 Varianza  = 3039.683594
27 StdDev    = 55.133327
28 Minimo    = -94.997849
29 Maximo    = 78.435913
30 Tiempo promedio del kernel (1 rep.): 32548.343086 ms
31 [Inferior 1 (process 27310) exited normally]

```

Listing 17: Transcripción de GDB: `x/8fw out` tras el bucle de normalización

```

steven-ubuntu@Ubuntu: ~/Proyecto-Arquitectura/proyecto_vectorial — gdb --args bin/...
~/Proyecto-Arquitectura/proyecto_vectorial

steven-ubuntu@Ubuntu:~/Proyecto-Arquitectura/proyecto_vectorial$ python3 tools/gen_input.py 16 data/input_gdb16.dat random 42
Generado 'data/input_gdb16.dat' con N=16, modo=random
steven-ubuntu@Ubuntu:~/Proyecto-Arquitectura/proyecto_vectorial$ gdb --args bin/norm_vector data/input_gdb16.dat data/output_gdb16.dat 1
GNU gdb (Ubuntu 17.1-2ubuntu1) 17.1
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from bin/norm_vector...
(gdb) break driver.c:171
Breakpoint 1 at 0x12fa: file src/driver.c, line 171.
(gdb) run data/input_gdb16.dat data/output_gdb16.dat 1
Starting program: /home/steven-ubuntu/Proyecto-Arquitectura/proyecto_vectorial/bin/norm_vector data/input_gdb16.dat data/output_gdb16.dat 1

This GDB supports auto-downloading debuginfo from the following URLs:
<https://debuginfod.ubuntu.com>
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, main (argc=<optimized out>, argv=<optimized out>) at src/driver.c:171
171      clock_gettime(CLOCK_MONOTONIC, &t1); // F: Final del programa (end)
(gdb) print out
$1 = (float *) 0x555555559520
(gdb) x/8fw out
0x555555559520: 0.925018966  -1.30381823  -0.396858305  -0.58483306
0x555555559530: 1.27705455  1.06022835  1.84189725  -1.07916975
(gdb) continue
Continuing.
N      = 16
Suma   = -369.824280
Media  = -23.114017
Varianza = 3039.683594
StdDev  = 55.133327
Minimo  = -94.997849
Maximo  = 78.435913
Tiempo promedio del kernel (1 rep.): 32548.343086 ms
[Inferior 1 (process 27310) exited normally]
(gdb)

```

Figura 3: Captura de la sesión de GDB donde se evidencia el contenido del arreglo de salida out tras el bucle de normalización vectorial ($N=16$).

8. Conclusiones, limitaciones y trabajo futuro

8.1. Conclusiones

1. En primer lugar, la vectorización con AVX2 cumple totalmente con el objetivo planteado, pero es importante destacar que fue de forma acotada debido a la arquitectura y no por el diseño ejecutado en el kernel. Asimismo, se logró una reducción de instrucciones cercana al factor teórico, el cual es de $8\times$ ($7,73-7,75\times$ medido con `perf stat`), confirmando de esta manera que ambas rutinas explotan correctamente el ancho de los registros utilizados, los registros YMM. Por otra parte, el speedup real osciló entre $3,7\times$ y $8,3\times$ según N , lo que a su vez, llega a demostrar que el paralelismo de datos por sí solo no garantiza el speedup teórico sin un análisis completo del resto de variables del sistema.
2. En segundo lugar, el cuello de botella se desplaza de cómputo a memoria a medida que crece N . Esto se visualiza, para arreglos de datos pequeños ($N = 10^3-10^5$), donde los datos del mismo caben perfectamente en L1/L2/L3 y a su vez, el kernel vectorial está limitado por cómputo y se acerca al techo de $8\times$. De la forma similar, a partir de arreglos con un $N = 10^6$, donde los mismo superan la capacidad de la L3 (4 MB) y el ancho de banda de memoria pasa a

ser el factor dominante, puesto que el IPC vectorial cae de 0,90 a 0,58 mientras que el de la versión escalar se logra mantener estable ($\sim 1,47$ – $1,48$). Finalmente, el rendimiento efectivo del sistema de memoria de la versión vectorial baja de ~ 35 GB/s a ~ 14 GB/s, validando de esta manera la Ley de Amdahl aplicada al ancho de banda.

3. En tercer lugar, las decisiones de codificación no fueron relacionadas con el ancho vectorial puesto que también contribuyeron al speedup medido. Asimismo, el uso de `vminps/vmaxps` sin saltos condicionales (frente a `comiss+jae/jbe` en la versión escalar) eliminó ramas dependientes de datos del bucle principal, y la codificación de tipo VEX de tres operandos evitó instrucciones `movaps` adicionales necesarias con la versión escalar - SSE. De la misma forma, el speedup observado no es atribuible únicamente al paralelismo SIMD de 8 carriles, sino también a la eliminación de predicción de saltos y a la estructura no destructiva de la versión escalar AVX2.
4. En cuarto lugar, los costos fijos (como lo son el prólogo/epílogo, la reducción horizontal y la llamadas entre las diversas rutinas) explican el speedup óptimo en arreglos de datos pequeños, puesto que con $N = 10^3$ el speedup fue de solamente $6,7\times$ aunque todo el trabajo efectuado reside en L1, puesto que las cadenas de las distintas reducciones horizontales (`vextractf128 + vaddps + dos vhaddps`) no se pueden pagar con pocas iteraciones realizada. Finalmente, la parte no vectorizable del kernel, aunque la misma es relativamente pequeña, pesa proporcionalmente más cuanto menor es N .
5. En quinto y último lugar, la verificación funcional realizada con la herramienta de GDB y la de correctud numérica (casos borde vs. NumPy) respaldan que la ganancia de rendimiento no se obtuvo a costa de la exactitud, puesto que los 6 casos de prueba (incluyendo $N = 0$, $N = 1$, remanente no múltiplo de 8, varianza cero y valores extremos) coincidieron perfectamente con la referencia dentro de la tolerancia relativa de 10^{-4} y la inspección realizada de los diversos registros YMM en GDB confirmó que la reducción horizontal y el manejo del remanente se ejecutan exactamente como se diseñaron.

8.2. Limitaciones observadas

Es importante indicar que el presente proyecto presenta diversas limitaciones relevantes que se deben señalar, las cuales se muestran a continuación:

1. En primer lugar, la totalidad de las mediciones de rendimiento se obtuvo sobre un único procesador (AMD Ryzen 5 7520U), por lo que el speedup reportado en la Sección 6 no necesariamente se puede generalizar o comparar a otras microarquitecturas, como lo pueden ser la de los procesadores Intel o generaciones distintas de AMD, puesto que el comportamiento del *AVX offset* y del ancho de banda de memoria puede cambiar o variar considerablemente.
2. En segundo lugar, se tiene que ambos kernels operan sobre el tipo de dato `float` que es de 32 bits y la comparación contra la referencia en NumPy (la cual emplea `double`) exige una tolerancia relativa de 10^{-4} descrita en la Sección 6 del presente proyecto, lo cual a su vez, puede llegar a esconder la acumulación natural del error de redondeo en sumas de los arreglos que son muy grandes ($N = 5 \times 10^7$).
3. En tercer lugar, la ruta de contingencia hacia SSE/AVX que se valoró en el enunciado del proyecto nunca se ejecutó ni se validó, puesto que el hardware disponible soportó AVX2 de forma directa.

Finalmente, se tiene que tanto la versión escalar como la vectorial conservaron deliberadamente la instrucción de división (`divss/vdivps`) sin sustituirla en ningún momento por el cálculo del recíproco, y la carga de datos en las rutinas `sum_array` y `compute_stats` que se realizó mediante el uso de la instrucción `vmovups` en lugar de la instrucción `vmovaps` pese a contar con la alineación de 32 bytes (Sección 4.8), por lo que a su vez produce que el speedup medido no representa el máximo de rendimiento alcanzable, sino específicamente el efecto aislado de la vectorización SIMD.

8.3. Trabajo futuro

En primer lugar, se propone que sería relevante sustituir la división que se ejecuta con la instrucción (`vdivps`) en la rutina `normalize_array` por el cálculo del recíproco de σ mediante la instrucción `vrcpps` y a su vez reemplazar las cargas dadas por la instrucción `vmovups` por la instrucción `vmovaps` donde la alineación de 32 bytes ya está garantizada, esto con el fin de aislar y de cuantificar la ganancia adicional que estas dos optimizaciones podrían aportar sobre el speedup ya obtenido y analizado. En segundo lugar, se recomendaría el poder volver a repetir las mediciones descritas en la Sección 6 con la utilización de diversos procesadores Intel y sobre otras generaciones de AMD, para de esta manera poder determinar si el comportamiento observado en cuanto al ancho de banda de memoria y el *AVX offset* se generaliza a otras microarquitecturas x86-64. Finalmente, es importante indicar que se plantea explorar el conjunto de instrucciones de AVX-512 en un hardware compatible, así como combinar la vectorización SIMD ya implementada con paralelismo a nivel de hilos (por ejemplo mediante el uso de la herramienta de OpenMP o `pthread`s) para arreglos de tamaño considerablemente mayor al $N = 5 \times 10^7$ probado en este proyecto.

A. Referencia al diagrama de arquitectura (entregable 3.a)

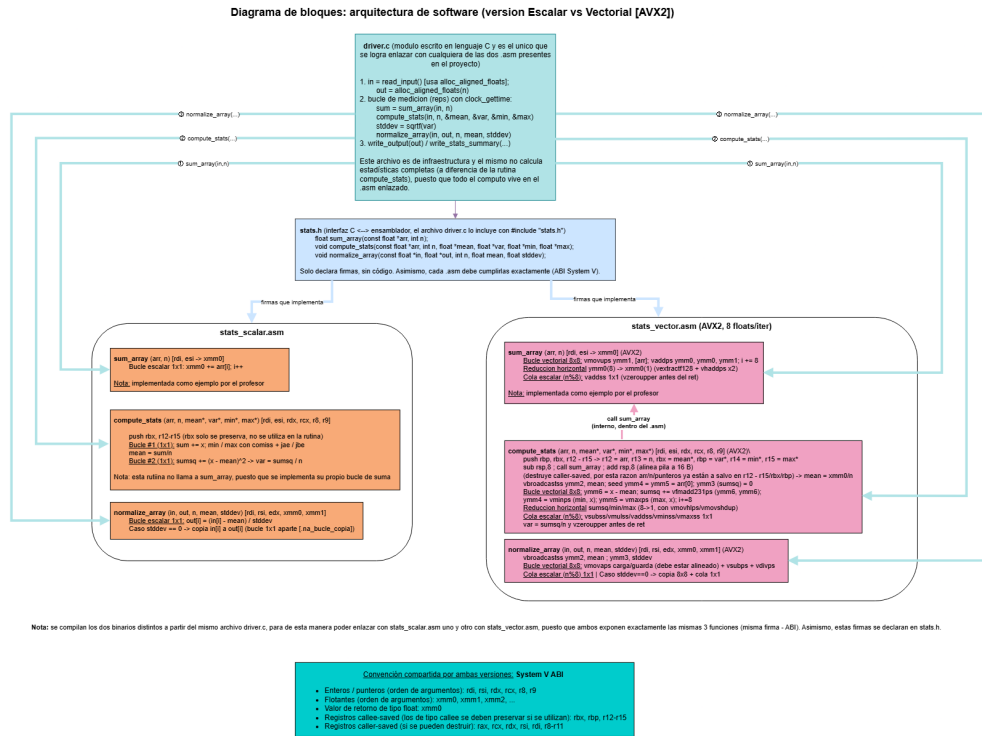


Figura 4: Diagrama de bloques: arquitectura de software (versión Escalar vs Vectorial AVX2)

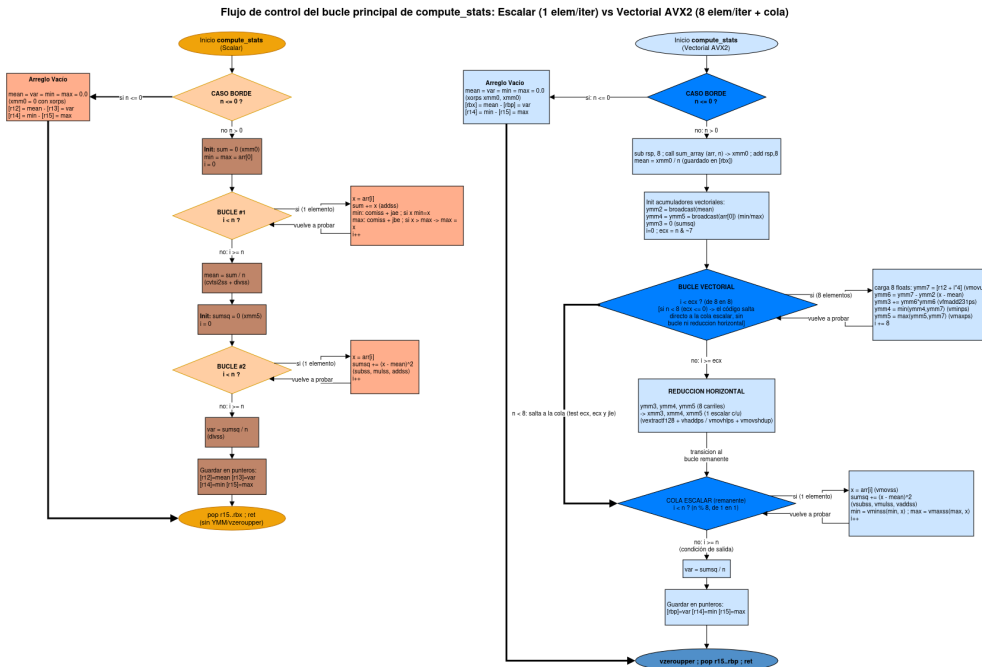


Figura 5: Flujo de control del bucle principal de la rutina compute_stats: Versión Escalar (1 elem/iter) vs Versión Vectorial AVX2 (8 elem/iter + cola)

Flujo de control de sum_array: Escalar (1 elem/iter) vs Vectorial AVX2 (8 elem/iter + cola)

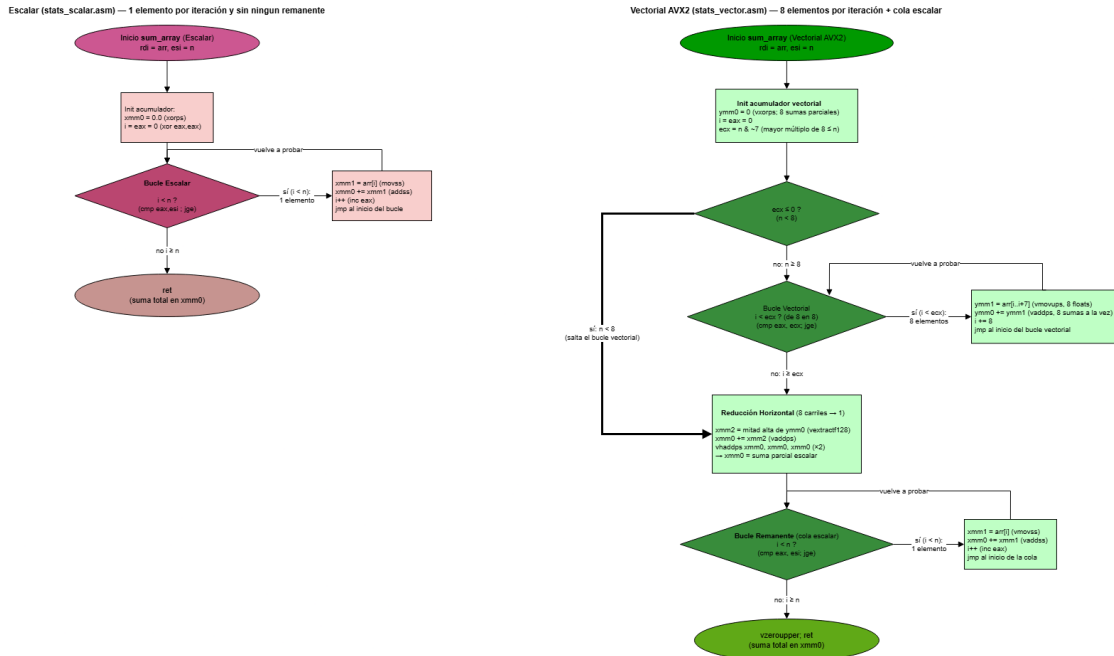


Figura 6: Flujo de control de sum_array: Versión Escalar (1 elem/iter) vs Versión Vectorial AVX2 (8 elem/iter + cola)

Flujo de control de normalize_array: Escalar (1 elem/iter) vs Vectorial AVX2 (8 elem/iter + cola)

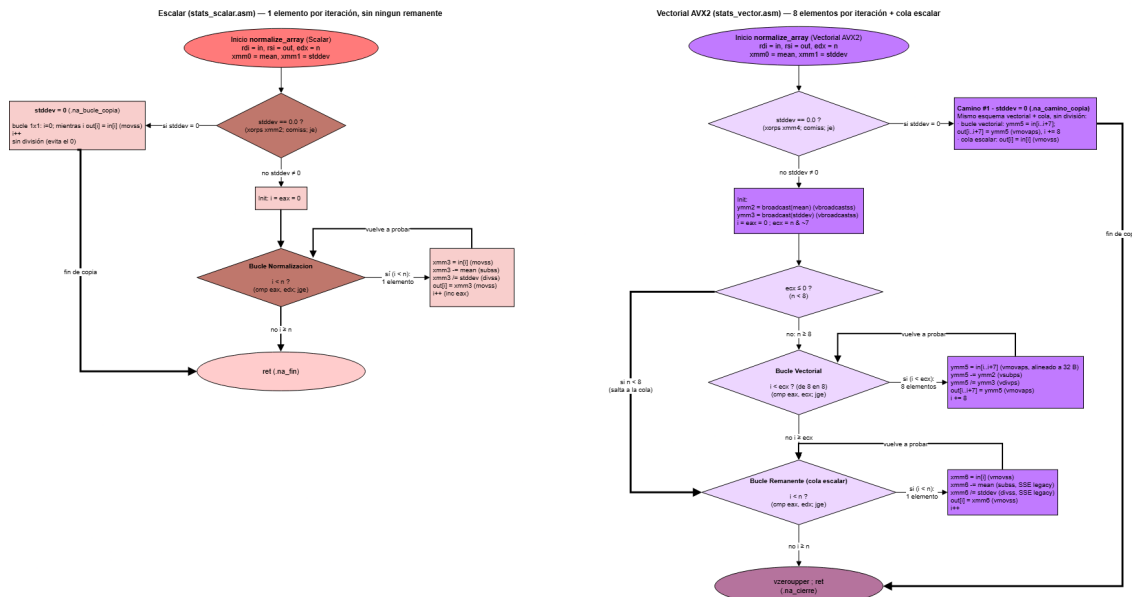


Figura 7: Flujo de control de normalize_array: Versión Escalar (1 elem/iter) vs Versión Vectorial AVX2 (8 elem/iter + cola)

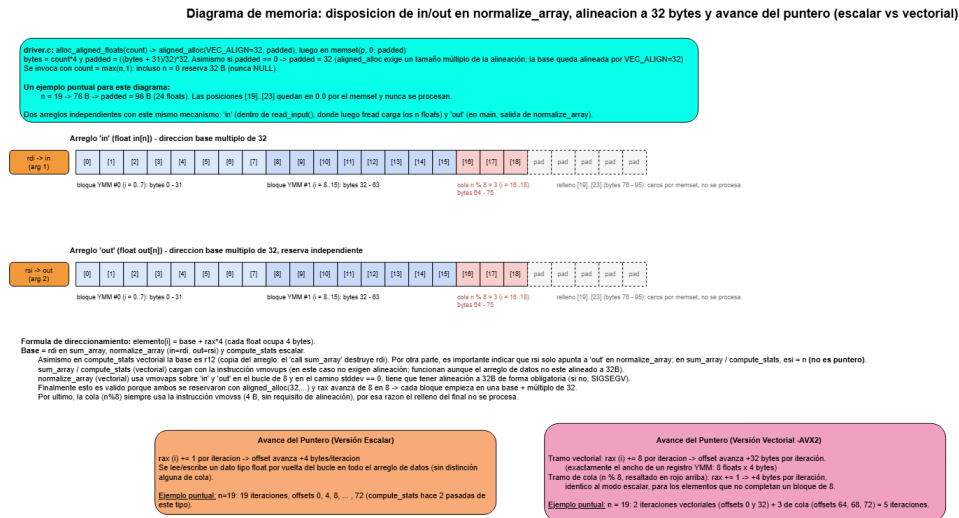


Figura 8: Diagrama de memoria: disposición de in/out en normalize_array, alineación a 32 bytes y avance del puntero (versión escalar vs versión vectorial)

B. Pasos para correr el proyecto

En primer lugar se corren las siguientes líneas una única vez:

1. sudo apt update
2. sudo apt install -y nasm gcc make python3
3. lscpu | grep -o avx2 | head -1 (Debe imprimir AVX2 para verificar la compatibilidad del CPU)
4. git clone https://github.com/steven-loaiza-cr/Proyecto-Arquitectura.git

Continuamente se deben correr las siguientes instrucciones en la consola de linux:

1. cd /ruta/a/Proyecto-Arquitectura/proyecto_vectorial
2. make clean
3. make
4. python3 tools/gen_input.py 1000 data/input.dat random 42 (El número 1000 es el tamaño del arreglo de datos - Se debe cambiar si se quiere otro tamaño)
5. make run-scalar
6. make run-vector
7. python3 tools/verify_reference.py data/input.dat data/output_scalar.dat.stats.txt
8. python3 tools/verify_reference.py data/input.dat data/output_vector.dat.stats.txt

C. Link del GitHub

<https://github.com/steven-loaiza-cr/Proyecto-Arquitectura.git>

Referencias

- [Bryant and O'Hallaron, 2016] Bryant, R. E. and O'Hallaron, D. R. (2016). *Computer Systems: A Programmer's Perspective*. Pearson, 3 edition.
- [Intel Corporation, 2026] Intel Corporation (2026). *Intel® 64 and IA-32 Architectures Software Developer's Manual*. Combined Volumes: 1, 2A, 2B, 2C, 2D, 3A, 3B, 3C, 3D, and 4. Order Number: 325462-092US.
- [Patterson and Hennessy, 2019] Patterson, D. A. and Hennessy, J. L. (2019). *Computer Architecture: A Quantitative Approach*. Morgan Kaufmann, 6 edition.